



Projeto prático de banco de dados

Prof. Sidney Nicolau Venturi Filho

Apresentação

Você vai conhecer os aspectos práticos do projeto de um banco de dados. Também verá como realizar modelagem, consultar dados utilizando a linguagem SQL e programar em um SGBD com uma linguagem procedural.

Propósito

Objetivos

Módulo 1

Modelando e criando o banco de dados do museu

Formular o banco de dados do museu no PostgreSQL.

Módulo 2

Consultando dados em uma tabela

Aplicar consultas SQL para uma tabela.

Módulo 3

Comandos SELECT com duas ou mais tabelas

Aplicar consultas SQL para mais de uma tabela.

Módulo 4

Linguagem procedural de banco de dados

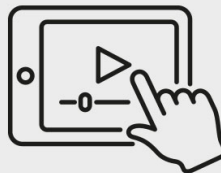
Programar utilizando PL/pgSQL.



Introdução

Assista ao vídeo para conferir uma apresentação sobre o projeto prático de banco de dados.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material



1 - Modelando e criando o banco de dados do museu

Ao final deste módulo, você será capaz de formular o banco de dados do museu no PostgreSQL.

Ligando os pontos

Imagine que você foi contratado para modelar e criar o banco de dados de uma startup chamada EduTech, que oferece soluções de tecnologia para instituições de ensino. Atualmente, a empresa está desenvolvendo um sistema de gerenciamento para acompanhar informações sobre os cursos oferecidos e os alunos matriculados.

A modelagem e a implementação do banco de dados devem atender aos seguintes requisitos:

- **Cursos** – Armazenar informações sobre os cursos oferecidos pela EduTech. Os atributos incluirão ID do curso, nome do curso, descrição, carga horária e professor responsável.
- **Alunos** – Representar os alunos matriculados nos cursos. Os atributos incluirão ID do aluno, nome, e-mail, data de matrícula e informações de contato.
- **Matrículas** – Registrar as matrículas dos alunos nos cursos. Os atributos incluirão ID da matrícula, ID do aluno, ID do curso, data de matrícula e *status* da matrícula (ativa ou cancelada).

Observe esses relacionamentos:

- Aluno está matriculado em curso – um aluno pode estar matriculado em vários cursos ao mesmo tempo. Um curso pode ter vários alunos matriculados.
- Curso possui matrícula de aluno – cada curso pode ter várias matrículas de alunos ao longo do tempo. Cada matrícula está associada a apenas um curso e um aluno.

Após a leitura do case, é hora de aplicar seus conhecimentos! Vamos ligar esses pontos?

Questão 1

Qual dos seguintes aspectos é mais importante considerar ao projetar a modelagem lógica do banco de dados da EduTech?

- A O número total de cursos oferecidos pela EduTech.
- B A relação entre cursos, alunos e matrículas.
- C A descrição detalhada de cada curso.
- D A data de matrícula de cada aluno.
- E O nome do professor responsável por cada curso.

Parabéns! A alternativa B está correta.

Ao projetar a modelagem lógica do banco de dados da EduTech, a relação entre cursos, alunos e matrículas é um aspecto importante a ser considerado. Essa relação determina como os cursos são associados aos alunos matriculados e como as informações sobre as matrículas são armazenadas e gerenciadas.

Questão 2

Qual dos seguintes princípios de mapeamento entidade/tabela é mais relevante ao projetar o banco de dados da EduTech?

- A Cada atributo de uma entidade deve ser representado por uma coluna em uma tabela.
- B Cada tabela deve ser normalizada para reduzir redundância de dados.
- C Cada entidade deve ser mapeada para múltiplas tabelas para aumentar a modularidade.
- D Cada relação de muitos-para-muitos deve ser resolvida por uma tabela de junção.
- E Cada tabela deve conter uma chave primária única para identificar exclusivamente cada registro.

Parabéns! A alternativa A está correta.

Ao projetar o banco de dados da EduTech, é essencial seguir o princípio de que cada atributo de uma entidade deve ser representado por uma coluna em uma tabela. Isso garante que todos os atributos de uma entidade sejam adequadamente capturados e armazenados no banco de dados, contribuindo para a integridade e consistência dos dados.

Questão 3

Descreva os principais componentes da modelagem conceitual que você consideraria ao projetar o banco de dados da EduTech. Explique como você representaria esses componentes e discuta a importância de cada um deles para o sucesso do projeto.

Digite sua resposta aqui

Chave de resposta ▾

Ao projetar o banco de dados da EduTech, diversos componentes da modelagem conceitual devem ser considerados:

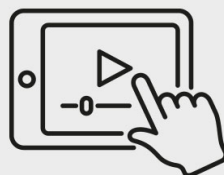
- Entidades e atributos: para representar os objetos do mundo real e suas características.
- Relacionamentos: para representar as associações do mundo real.
- Cardinalidade: para definir como os objetos se relacionam.
- Chaves primárias e estrangeiras: para impor as restrições de entidade e referencial.

Realizando a modelagem conceitual

O caso do museu servirá de base ao longo deste estudo. O ponto de partida será a modelagem conceitual, lógica e física até sua implementação no PostgreSQL, incluindo a elaboração e execução de consultas.

Neste vídeo, abordaremos a seguinte etapa do projeto de banco de dados: modelagem conceitual aplicada ao minimundo museu.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Minimundo do museu

Um museu deseja automatizar o controle de seu acervo e de algumas atividades administrativas. Durante o levantamento de requisitos, os seguintes dados foram obtidos:

- Um autor possui um código de autor que identifica seu nome e sua nacionalidade.
- Os autores produzem obras identificadas por um código. Essas obras possuem um título e o ano de sua produção.
- As informações dos autores que um dia foram expostos no museu são conservadas mesmo quando suas obras são retiradas do acervo, visando facilitar o cadastro de novas obras deles que venham a entrar em exposição.
- As obras podem ser pinturas ou esculturas.
- Se forem esculturas será necessário registrar sua altura, peso e material.
- Se forem pinturas será necessário registrar o estilo e a área.
- Quando uma obra é retirada do acervo, seus dados são apagados do banco de dados.
- As obras são expostas em salões.
- Cada salão é identificado por um número, possui uma área e está localizado em um determinado andar.
- Os funcionários do museu são identificados por seu CPF, possuem um nome, salário, turno e função.
- Os turnos existentes são manhã, tarde ou noite, codificados como M, T e N, respectivamente.
- As funções existentes são faxineiro ou segurança.
- Os funcionários são alocados em salões durante um determinado período de seu turno, com hora de entrada e saída do local pré-estabelecida.

Modelando as entidades e seus atributos

Um **autor** possui um código de autor que identifica seu nome e sua nacionalidade.

Entidade autor.

Os autores produzem **obras** que são identificadas por um código. Elas possuem um título e o ano de sua produção.

As obras podem ser **pinturas** ou **esculturas**. Se forem esculturas, será necessário registrar sua altura, seu peso e seu material. Se forem pinturas, será necessário registrar o seu estilo e a área.

Entidades obra, pintura e escultura.

As obras são expostas em salões. Cada **salão** é identificado por um número, possui uma área e está localizado em um determinado andar.

Entidade salão.

Cada **funcionário** do museu é identificado por seu CPF, pelo nome, além de salário, turno e função.

Entidade funcionário.

Modelando os relacionamentos e seus atributos

Os **autores produzem obras**. Podemos notar no enunciado do minimundo que existe um relacionamento de PRODUCAO entre AUTOR e OBRA. Quanto à cardinalidade, devemos observar os seguintes aspectos do enunciado:

- As informações dos autores que um dia foram expostos no museu são conservadas mesmo quando suas obras são retiradas do acervo, visando facilitar a cadastro de novas obras deles que venham a entrar em exposição.
- Quando uma obra é retirada do acervo, seus dados são apagados do banco de dados.

Observe a imagem a seguir:

Relacionamento produção.

As obras são expostas em salões. A imagem seguinte mostra o relacionamento EXPOSICAO entre OBRA e SALAO.

Relacionamento exposição.

Os **funcionários são alocados em salões**. A frase destacada mostra a existência do relacionamento ALOCACAO entre FUNCIONARIO e SALAO.

Relacionamento alocação.

Quanto aos atributos de relacionamento, temos: os funcionários são alocados em salões durante um determinado período de seu turno, com **hora de entrada e saída** do local pré-estabelecido. Veja que o relacionamento ALOCACAO possui dois atributos, HORA_ENTRADA e HORA_SAIDA.

Atributos do relacionamento alocação.

Modelagem conceitual do museu

Finalizada a modelagem conceitual, confira a seguir o DER completo.

Modelo conceitual do minimundo museu.

Atividade 1

Ao projetar um banco de dados, uma das tarefas cruciais é organizar e modelar as informações de forma eficiente e significativa. Quando devemos utilizar estruturas de generalização/especialização?

- A Ao criar uma hierarquia de classes em programação orientada a objetos.
- B Ao projetar um banco de dados relacional para armazenar informações de diferentes tipos de entidades com características comuns e distintas.
- C Ao otimizar o desempenho de consultas em um banco de dados NoSQL.
- D Somente ao criar diagramas de casos de uso em engenharia de software.
- E Nunca, pois estruturas de generalização/especialização são obsoletas em sistemas modernos de gerenciamento de dados.

Parabéns! A alternativa B está correta.

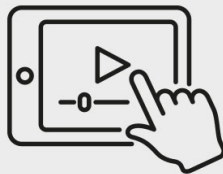
O uso de estruturas se deve ao fato de que a estrutura de generalização/especialização é particularmente útil em cenários em que existam entidades relacionadas que possuem atributos em comum, mas também características distintas que as diferenciem umas das outras.

Realizando a modelagem lógica

Agora que fizemos a modelagem conceitual, faremos a modelagem lógica.

Neste vídeo, abordaremos a etapa modelagem lógica aplicada ao minimundo museu do projeto de banco de dados.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Modelagem lógica das entidades

A regra básica para esse tipo de modelagem é que cada entidade gera uma tabela e cada atributo gera uma coluna. Teremos de definir o tipo de dado de cada coluna e as eventuais restrições.

ENTIDADE	TABELA	COLUNA	
AUTOR	AUTORES	CODIGO	
		NOME	
		NACIONALIDADE	
SALAO	SALOES	NUMERO	
		ANDAR	

ENTIDADE	TABELA	COLUNA	
		AREA	↑
FUNCIONARIO	FUNCIONARIOS	CPF	
		NOME	
		SALARIO	↑
		TURN0	
		FUNCAO	

Tabelas AUTOR, SALAO e FUNCIONARIO.

Do ponto de vista de modelo, seria da seguinte forma:

Tabelas AUTOR, SALAO e FUNCIONARIO.

Modelagem lógica da generalização/especialização

O primeiro passo é mapear cada entidade da estrutura para uma tabela e cada atributo para uma coluna, como o tipo e as restrições cabíveis.

ENTIDADE	TABELA	COLUNA	TIPO
OBRA	OBRAS	CODIGO	INTEIRO
		ANO	INTEIRO
		TITULO	TEXTO(50
ESCULTURA	ESCULTURAS	MATERIAL	TEXTO(30
		ALTURA	NUMERICO(
		PESO	NUMERICO(
PINTURA	PINTURAS	ESTILO	TEXTO(30

ENTIDADE	TABELA	COLUNA	TIPO
		AREA	NUMERICO(

Tabelas OBRAS, ESCULTURAS E PINTURAS.

Do ponto de vista de modelo, seria da seguinte forma:

Tabelas OBRAS, ESCULTURAS e PINTURAS.

É preciso implementar a estrutura no modelo. Para isso, as tabelas PINTURAS e ESCULTURAS recebem a chave primária de OBRAS como suas respectivas chaves primárias, sendo também chaves estrangeiras. O modelo final da generalização/especialização ficaria como na imagem seguinte.

Tabelas OBRAS, ESCULTURAS e PINTURAS.

Modelagem lógica dos relacionamentos

Vamos iniciar com os relacionamentos PRODUCAO e EXPOSICAO, que são do tipo 1:N. A solução típica é levar a chave primária das tabelas do lado 1 (AUTOR e SALAO) como chave estrangeira para a tabela do lado N (OBRAS).

Identificação de chaves estrangeiras.

A modelagem fica da seguinte forma:

Tabelas AUTORES, OBRAS e SALOES.

Para relacionamento ALOCACAO, que é do tipo N:N, utilizaremos a solução-padrão. Essa solução consiste em criar uma tabela para o relacionamento.

Identificação de chaves estrangeiras.

O modelo ficaria então desta forma, sendo a PK de ALOCACOES composta de dois atributos:

Tabelas SALOES, ALOCACOES e FUNCIONARIOS.

Modelando a restrição de integridade

Os turnos existentes são manhã, tarde ou noite, codificados como M, T e N, respectivamente. Adotaremos uma tabela de valores para o turno.

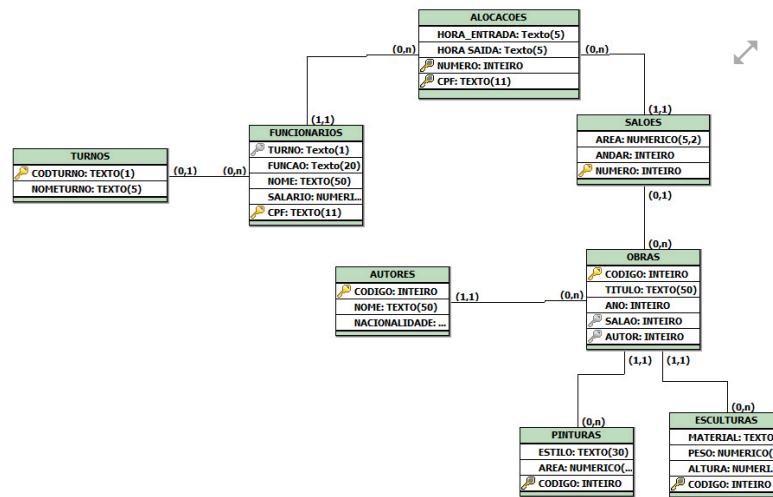
Vamos então criar a tabela TURNO com duas colunas:

- CODTURNO – Texto(1) chave primária.
- NOMETURNO – Texto(5) não nulo.

Observe a imagem a seguir.

Tabelas TURNOS e FUNCIONARIOS.

O modelo lógico final seria o seguinte:



Modelo lógico final.

Atividade 2

Durante a modelagem lógica do caso do museu, uma série de decisões foram tomadas quanto às chaves das diversas tabelas, dentre elas podemos afirmar que a chave primária de PINTURAS corresponde à chave estrangeira recebida de OBRAS.

Porque

Como a entidade PINTURA é uma especialização da entidade OBRA, ela herda a chave primária da entidade generalizada.

Quanto às afirmativas, é correto dizer que:

- A As duas afirmações estão corretas, e a segunda justifica a primeira.
- B As duas afirmações estão corretas, e a segunda não justifica a primeira.
- C A primeira afirmação está correta, e a segunda está incorreta.
- D A primeira afirmação está incorreta, e a segunda está correta.
- E As duas afirmações estão incorretas.

Parabéns! A alternativa A está correta.

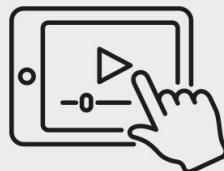
Para realizarmos a modelagem lógica de uma estrutura de generalização/especialização, uma das soluções possíveis é criar uma tabela para que cada entidade seja generalizada ou especializada. Nesse caso, a PK da especializada será a PK da generalizada que ela herda e que ainda será um FK para a tabela da generalizada.

Modelagem física e implementação do banco de dados

Vamos implementar o banco de dados no PostgreSQL. Para isso, precisamos mapear os tipos genéricos especificados na modelagem lógica para os tipos efetivamente existentes no SGBD, bem como estabelecer as restrições necessárias e devemos criar os comandos SQL necessários.

Neste vídeo, abordaremos a etapa do projeto físico de banco de dados aplicada ao minimundo museu.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Tipos de dados do PostgreSQL

Vejamos o mapeamento dos dados genéricos para os dados específicos do PostgreSQL:

TABELA	COLUNA	TIPO LÓGICO
AUTORES	CODIGO	INTEIRO
	NOME	TEXT(50)
	NACIONALIDADE	TEXT(30)
SALOES	NUMERO	INTEIRO
	ANDAR	INTEIRO
	AREA	NUMERICO(5,2)
FUNCIONARIOS	CPF	TEXT(11)
	NOME	TEXT(50)
	SALARIO	NUMERICO(7,2)
	TURNIO	TEXT(1)
	FUNCAO	TEXT(20)

TABELA	COLUNA	TIPO LÓGICO	
OBRAS	CODIGO	INTEIRO	
	ANO	INTEIRO	
	TITULO	TEXT0(50)	
	AUTOR	INTEIRO	
	SALAO	INTEIRO	
ESCULTURAS	MATERIAL	TEXT0(30)	
	ALTURA	NUMERICO(5,2)	
	PESO	NUMERICO(6,2)	
	CODIGO	INTEIRO	
PINTURAS	ESTILO	TEXT0(30)	
	AREA	NUMERICO(5,2)	
	CODIGO	INTEIRO	
TURNOS	CODTURNO	TEXT0(1)	
	NOMETURNO	TEXT0(5)	
ALOCACAO	CPF	TEXT0(11)	
	NUMERO	INTEIRO	

TABELA	COLUNA	TIPO LÓGICO	
	HORA_ENTRADA	TEXTO(5)	
	HORA_SAIDA	TEXTO(5)	

Tipos de dados do PostgreSQL.

Gerenciando as tabelas no banco de dados

Criando tabelas

O comando de criação de tabela é o **CREATE TABLE**.

Veja a sintaxe:

LINGUAGEM SQL



Código em SQL.

Alterando tabelas

Para alterarmos a estrutura de uma tabela, o comando é **ALTER TABLE**.

Observe a sintaxe correspondente:

LINGUAGEM SQL



Código em SQL.

Eliminando tabelas

Para eliminar uma tabela, o comando é **DROP**, cuja sintaxe é a seguinte:

LINGUAGEM SQL



Código em SQL.

Para apagar as tabelas referenciadas em chaves estrangeiras, é necessário apagar primeiro as tabelas que as referenciam.

Criando as tabelas do museu

Para preparar os comandos de criação das tabelas, são necessários alguns cuidados. As tabelas que possuem chave estrangeira para outras tabelas devem ser criadas depois das tabelas das chaves primárias, ou então as chaves devem ser criadas após a criação de todas as tabelas. No script deste estudo, utilizaremos a segunda opção.

Veja os comandos de criação de tabelas e das chaves estrangeiras:

<pre>CREATE TABLE PINTURAS (ESTILO VARCHAR(30), AREA NUMERIC(5,2), CODIGO INTEGER PRIMARY KEY);</pre>	<pre>CREATE TABLE ESCULTURAS (MATERIAL VARCHAR(30), PESO NUMERIC(6,2), ALTURA NUMERIC(5,2), CODIGO INTEGER PRIMARY KEY);</pre>
<pre>CREATE TABLE OBRAS (CODIGO INTEGER PRIMARY KEY,</pre>	<pre>CREATE TABLE ALOCACOES (HORA_ENTRADA CHAR(5),</pre>

TITULO VARCHAR(50), ANO INTEGER, SALAO INTEGER, AUTOR INTEGER);	HORA_SAIDA CHAR(5), NUMERO INTEGER, CPF CHAR(11), PRIMARY KEY(NUMERO,CPF));
CREATE TABLE AUTORES (CODIGO INTEGER PRIMARY KEY, NOME VARCHAR(50), NACIONALIDADE VARCHAR(50));	CREATE TABLE SALOES (AREA NUMERIC(5,2), ANDAR INTEGER, NUMERO INTEGER PRIMARY KEY);
CREATE TABLE TURNOS (CODTURNO CHAR(1) PRIMARY KEY, NOMETURNO VARCHAR(5));	
CREATE TABLE FUNCIONARIOS (TURNO CHAR(1), FUNCAO VARCHAR(20), NOME VARCHAR(50), SALARIO NUMERIC(7,2), CPF CHAR(11) PRIMARY KEY, CONSTRAINT VALIDAFUNCAO CHECK (FUNCAO IN ('segurança', 'faxineiro')));	
ALTER TABLE PINTURAS ADD FOREIGN KEY(CODIGO) REFERENCES OBRAS (CODIGO); ALTER TABLE ESCULTURAS ADD FOREIGN KEY(CODIGO) REFERENCES OBRAS (CODIGO); ALTER TABLE OBRAS ADD FOREIGN KEY(SALAO) REFERENCES SALOES (NUMERO); ALTER TABLE OBRAS ADD FOREIGN KEY(AUTOR) REFERENCES AUTORES (CODIGO); ALTER TABLE ALOCACOES ADD FOREIGN KEY(NUMERO) REFERENCES SALOES (NUMERO); ALTER TABLE ALOCACOES ADD FOREIGN KEY(CPF) REFERENCES FUNCIONARIOS (CPF); ALTER TABLE FUNCIONARIOS ADD FOREIGN KEY(TURNO) REFERENCES TURNOS(CODTURNO);	

Script de criação das tabelas.

[Obtenha o script](#) para criar as tabelas.

Manipulando os dados do banco

Inserindo dados em uma tabela

O comando **INSERT** insere linhas em uma tabela.

A sintaxe básica desse comando é a seguinte:

LINGUAGEM SQL



Código em SQL.

Alterando dados de uma tabela

O comando **UPDATE** permite atualizar dados já existentes nas tabelas.

A sintaxe é a seguinte:

LINGUAGEM SQL



Código em SQL.

Por exemplo, para alterar o nome da região 1 para EXEMPLO1, o comando é o seguinte:

LINGUAGEM SQL



Código em SQL.

Eliminando dados de uma tabela

O comando **DELETE** é utilizado para excluir linhas em uma tabela. Veja a sintaxe desse comando:

LINGUAGEM SQL



Código em SQL.

Para apagar todas as linhas de REGIAO, o comando é o seguinte:

LINGUAGEM SQL



Código em SQL.

Controlando transações

Uma transação é um conjunto de operações no banco de dados. Para efetivarmos as operações realizadas pelo comando, devemos utilizar o comando **COMMIT**. Por outro lado, se desejamos desfazer as operações, devemos comandar **ROLLBACK**.

No caso do PostgreSQL, que utilizaremos em nossas práticas, o pgAdmin, ferramenta de administração que utilizaremos para realizar os

comandos, vem configurado para **AUTO COMMIT**, ou seja, efetiva imediatamente os comandos após a sua execução.

Commit no pgAdmin.

Ao desmarcar a opção apresentada na imagem, será necessário comandar COMMIT após as operações CREATE, ALTER, DROP, INSERT, UPDATE e DELETE.

Inserindo dados no banco do museu

Veja no [Script dos dados](#) os comandos necessários para inserir os dados no museu.

Atividade 3

A linguagem SQL fornece uma série de comandos que possibilita a criação de objetos e a manipulação de dados. O comando que permite fazer alterações nas linhas de uma tabela é o:

- A UPDATE
- B ALTER
- C UPGRADE
- D DROP
- E ALTER DATA

Parabéns! A alternativa A está correta.

Os comandos para manipulação de dados que compõem a DML são INSERT, DELETE e UPDATE. No entanto, o comando UPDATE permite

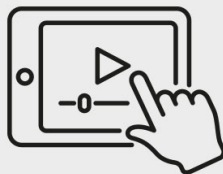
inserir linhas em uma tabela.

Criando o banco de dados do museu no PostgreSQL

Vamos criar o banco de dados do museu que utilizaremos nas práticas restantes. Demonstraremos a instalação do servidor e o passo a passo da criação do banco. Você pode baixar o [script da criação das tabelas](#) e a inserção das linhas no banco de dados da empresa.

Acompanhe, neste vídeo, todos os detalhes da implementação do banco de dados do MUSEU no PostgreSQL. Não perca!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para criar o banco de dados da empresa, obedeça os seguintes passos:

1. Acesse o site oficial do PostgreSQL em <https://www.postgresql.org/>.
2. Selecione a versão mais recente compatível com o seu sistema operacional (Windows) e faça o download do instalador.
3. Execute o arquivo baixado para iniciar o instalador. Siga as instruções na tela para configurar o PostgreSQL.
4. Durante a instalação, será solicitada a configuração de um cluster. Defina uma senha para o superusuário (postgres) e configure as opções conforme necessário.
5. Escolha a porta padrão (geralmente 5432) e os diretórios para armazenamento de dados e arquivos de configuração.
6. Complete o processo de instalação e, se necessário, reinicie o sistema.
7. Instale a ferramenta de administração PgAdmin (opcional).
8. Abra o PGAdmin4 e faça conexão no servidor.

9. Crie um banco de dados chamado MUSEU.
10. Abra uma janela de consulta.
11. Carregue o script na janela de consulta.
12. Execute o script.
13. Valide a criação das tabelas na aba tabelas do banco da empresa.

Observe a imagem a seguir.

Visualização das tabelas usando o pgAdmin.

Atividade 4

Você foi designado para configurar um ambiente de desenvolvimento em um sistema Windows e precisa instalar e configurar o PostgreSQL como parte desse processo. Além da instalação do PostgreSQL, a equipe de desenvolvimento está explorando a utilização do Application Stack Builder (ASB) no PostgreSQL. Explique o propósito e a funcionalidade ASB nesse contexto.

Digite sua resposta aqui

Chave de resposta ▼

O Application Stack Builder (ASB) é uma ferramenta gratuita e de código aberto que auxilia na instalação e no gerenciamento de extensões para o PostgreSQL. Ele oferece uma interface gráfica amigável e simplifica o processo de:

1. **Instalar extensões** – Fornece uma lista completa de extensões disponíveis para o PostgreSQL, incluindo suas funcionalidades e requisitos, o que permite a instalação com um único clique.
2. **Gerenciar extensões** – Permite visualizar informações detalhadas sobre as extensões instaladas.

3. **Configurar extensões** – Fornece uma interface para configurar as opções de configuração das extensões instaladas.



2 - Consultando dados em uma tabela

Ao final deste módulo, você será capaz de aplicar consultas SQL para uma tabela.

Ligando os pontos

A empresa FashionWear é uma loja de roupas on-line que oferece uma variedade de produtos de moda para seus clientes. Para gerenciar suas operações e manter o controle sobre produtos, clientes e pedidos, a FashionWear utiliza um banco de dados relacional.

O banco de dados da FashionWear é composto por três tabelas principais:

- **Produtos** – contém informações sobre os produtos disponíveis na loja, incluindo ID do produto, nome, categoria, preço e quantidade em estoque.
- **Clientes** – armazena os detalhes dos clientes, como ID, nome, endereço, e-mail e número de telefone.
- **Pedidos** – registra todos os pedidos feitos pelos clientes, incluindo o ID do pedido, ID do cliente associado, ID do produto comprado, quantidade, preço unitário e data do pedido.

Agora, a equipe da FashionWear precisa realizar três consultas que envolvam apenas uma tabela cada, a fim de extrair informações úteis do banco de dados:

- **Consulta 1 - Produto mais caro** – para identificar o produto mais caro disponível na loja.
- **Consulta 2 - Clientes mais frequentes** – para determinar quem são os cinco clientes mais frequentes com base no número de pedidos feitos. É necessária uma consulta que conte o número de pedidos para cada cliente na tabela de pedidos e selecione os cinco clientes com o maior número de pedidos.
- **Consulta 3 - Produtos em estoque baixo** – para identificar todos os produtos que estão com estoque abaixo de um determinado limite, por exemplo, 10 unidades. É necessária uma consulta que selecione os produtos na tabela de produtos cuja quantidade em estoque é inferior a 10 unidades.

Após a leitura do case, é hora de aplicar seus conhecimentos! Vamos ligar esses pontos?

Questão 1

Qual seria a consulta SQL adequada para obter o maior preço de um produto na loja FashionWear?

- A `SELECT MAX(preco) FROM produtos.`
- B `SELECT preco FROM produtos ORDER BY preco DESC
LIMIT 1.`
- C `SELECT MAX(preco) AS maior_preco FROM produtos.`
- D `SELECT preco FROM produtos WHERE preco =
MAX(preco).`
- E `SELECT preco FROM produtos WHERE preco = TOP 1
preco.`

Parabéns! A alternativa A está correta.

Para obter o maior preço de um produto é necessário usar a função MAX para calcular o máximo valor da coluna de preços na tabela de produtos. Portanto, a opção correta é SELECT MAX(preco) FROM produtos. As outras opções não utilizam a função MAX adequadamente para essa finalidade.

Questão 2

Qual seria a consulta SQL adequada para identificar todos os produtos que estão com estoque abaixo de 10 unidades na FashionWear, conforme descrito no estudo de caso?

- A `SELECT * FROM produtos WHERE quantidade_estoque < 10.`
- B `SELECT * FROM produtos WHERE quantidade_estoque <= 10.`
- C `SELECT * FROM produtos WHERE quantidade_estoque > 10.`
- D `SELECT * FROM produtos WHERE quantidade_estoque = 10.`
- E `SELECT * FROM produtos WHERE quantidade_estoque BETWEEN 0 AND 10.`

Parabéns! A alternativa A está correta.

A consulta correta para identificar os produtos com estoque abaixo de um determinado limite é aquela que utiliza a cláusula WHERE para selecionar os registros em que a quantidade em estoque é menor que o limite estabelecido.

Questão 3

Descreva uma estratégia para determinar os cinco clientes mais frequentes na loja FashionWear com base no número de pedidos feitos. Explique como você escreveria a consulta SQL para alcançar esse objetivo e discuta as considerações importantes ao projetar essa consulta.

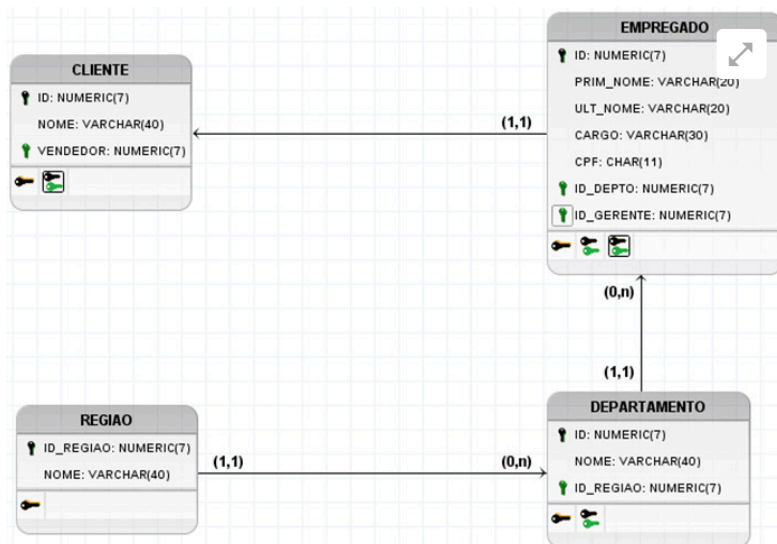
Digite sua resposta aqui

Chave de resposta ▾

A consulta busca identificar os clientes mais frequentes na loja FashionWear a partir do número de pedidos feitos por cada cliente na tabela de pedidos e da classificação em ordem decrescente com base nesse número. Dessa forma, os clientes que fizeram mais pedidos aparecerão no topo da lista. A consulta deve agrupar os pedidos pelo ID do cliente para contar o número de pedidos para cada cliente individualmente. Em seguida, a contagem de pedidos é ordenada de modo decrescente, o que permite que os clientes com mais pedidos apareçam primeiro. A consulta pode ser limitada para retornar apenas os cinco primeiros clientes com base nessa classificação, facilitando a identificação rápida dos clientes mais frequentes pela equipe da FashionWear e permitindo ações proativas para atendê-los melhor.

SELECT básico

Vamos iniciar o estudo do comando SELECT, aprendendo a retornar colunas das tabelas. Nos exemplos, utilizaremos o banco de dados da empresa criada anteriormente a partir do script e cujo modelo lógico está exibido a seguir:



Modelo lógico do banco de dados EMPRESA.

As tabelas do banco possuem os seguintes dados.

A tabela REGIAO contém informações geográficas.

Tabela REGIAO.

A tabela DEPARTAMENTO organiza as unidades da empresa.

Tabela DEPARTAMENTO.

A tabela EMPREGADO registra os funcionários.

Tabela EMPREGADO.

A tabela CLIENTE guarda dados dos clientes.

Tabela CLIENTE.

Tendo este banco em mente, é recomendável que você execute os comandos de exemplo no PostgreSQL. Para isso, crie o banco utilizando o script da [empresa](#).

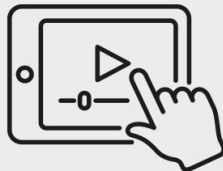
Você pode encontrar no arquivo [sqlmod2nc1](#) os comandos utilizados nos exemplos.



SELECT básico

Neste vídeo, abordaremos os comandos SELECT básicos da SQL no PostgreSQL utilizando a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Retornando colunas de uma tabela

O comando SQL que permite recuperar dados de uma tabela é o **SELECT**. A sintaxe básica desse comando é a seguinte:

LINGUAGEM SQL



Código SQL.

A seguir, veja o conteúdo da tabela EMPREGADO.

```
1 SELECT * FROM EMPREGADO;
```

	id [PK] numeric (7)	ut_nome character varying (20)	prim_nome character varying (20)	cargo character varying (30)	salario numeric (7,2)	dt_admissao date	cpf character (11)	id_depto numeric (7)	id_gerente numeric (7)
1	1	Velasques	Carmen	Presidente	29500.00	2009-05-05	34567890125	10	[null]
2	2	Neves	Lauro	Diretor de Compras	19500.00	2009-03-03	23456789012	30	1
3	3	Nogueira	Ermene	Diretor de Vendas	18000.00	2010-04-07	34567890123	20	1
4	4	Queiroz	Mark	Gerente de Compras	8000.00	2010-11-11	12345432123	30	2
5	5	Rodrigues	Alberto	Vendedor	4000.00	2008-10-10	87965432123	20	3
6	6	Ugarte	Marlene	Vendedor	3500.00	2009-03-03	87654345678	20	3

Consultando empregados.

Observe que não há nenhuma ordem ou seleção de linhas ou colunas. No **SELECT**, você também pode selecionar apenas algumas colunas e determinar a ordem em que retornam:

Consultando empregados.

Escrevendo expressões aritméticas em comandos SELECT

Em uma expressão, podemos especificar não apenas uma coluna, mas um dado derivado de uma ou mais colunas. Por exemplo, se desejarmos calcular o salário anual precedido pelo ID e PRIM_NOME do empregado, podemos escrever o seguinte comando:

Consultando salários de empregados.

Escrevendo expressões de concatenação

Concatenar implica juntar duas cadeias de caracteres. No PostgreSQL, o operador de concatenação é o `||`. Para retornar o nome completo dos empregados como uma única cadeia de caracteres, usamos o seguinte comando:

Consultando nomes dos empregados.

Observe que os nomes estão colados. Isso decorre do fato que depois do `PRIM_NOME` ou antes do `ULT_NOME` não existe espaço em branco armazenado na coluna. Para resolver isso, basta concatenar as colunas com um espaço em branco entre eles.

Consultando nomes dos empregados.

Criando alias de coluna

Quando são utilizadas expressões, o cabeçalho da coluna normalmente fica sem significado. Nesse caso, é necessário nomear as colunas de forma que mantenham o significado. Para isso, existem os alias de coluna.

Para criar um alias após a coluna, você deve utilizar `'AS'` seguido pelo título da coluna, entre aspas caso contenha espaço em branco, ou sem aspas se consistir em uma única palavra.

Criando alias de coluna.

Esteja atento, pois o `'AS'` é opcional. O comando funciona apenas com o alias depois do nome da coluna.

Criando alias de coluna.

Atividade 1

Observe a imagem a seguir e visualize o conteúdo da tabela EMPREGADO em um SGBD PostgreSQL:

Tabela empregado.

Em relação a essa tabela, foi executado um comando que produziu o seguinte resultado:

Consulta à tabela empregado.

O comando que produziu este resultado foi:

- A `SELECT ID, PRIM_NOME + ' '+ULT_NOME AS "NOME COMPLETO" FROM EMPREGADO.`
- B `SELECT ID, PRIM_NOME || ' '|| ULT_NOME AS 'Nome Completo' FROM EMPREGADO.`
- C `SELECT ID, PRIM_NOME || ' '|| ULT_NOME AS "Nome Completo" FROM EMPREGADO.`
- D `SELECT ID, PRIM_NOME || ' '|| ULT_NOME AS "NOME COMPLETO" FROM EMPREGADO.`
- E `SELECT ID, PRIM_NOME + ' '+ ULT_NOME AS "Nome Completo" FROM EMPREGADO.`

Parabéns! A alternativa C está correta.

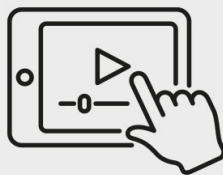
- O operador de concatenação do PostgreSQL é o ||, não o +.
- O alias deve vir entre aspas ("), não entre apostrofes (').
- O alias está grafado com a primeira letra de cada palavra em maiúsculas em vez de todas as letras.

Filtrando linhas na consulta

Até aqui, todos os comandos retornaram todas as linhas da tabela. Se você deseja filtrar as tabelas e retornar apenas as linhas que atendam a uma condição, é necessário acrescentar a cláusula WHERE ao comando SELECT. Você pode encontrar no arquivo [sqlmodnc2](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos os comandos SELECT com a cláusula WHERE da SQL no PostgreSQL utilizando a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Cláusula WHERE

Essa cláusula estabelece uma condição que a linha deverá obedecer para que faça parte do conjunto resposta da consulta. Nesse caso, retornam apenas as linhas que tenham verdadeiro como resposta para a condição. Veja o exemplo abaixo em que o objetivo é retornar apenas as linhas dos empregados com ID > 3:

```
SELECT *
FROM EMPREGADO
WHERE ID > 3;
```

id	ult_nome	prim_nome	cargo	salario	dt_admissao	cpf	id_depto	id_gerente
4	Queiroz	Mark	Gerente de Compras	8000.00	2010-11-11	12345432123	30	2
5	Rodrigues	Alberto	Vendedor	4000.00	2008-10-10	87965432123	20	3
6	Ugarte	Marlene	Vendedor	3500.00	2009-03-03	87654345678	20	3

Consultando empregados.

Repare que das seis linhas que a tabela possui, empregado com ID de 1 até 6, retornam apenas as três que possuem ID maior que 3 (4, 5 e 6). Na construção das condições, você pode utilizar os seguintes operadores relacionais: =, <>, >, <,>= e <=.

Atenção!

É importante estar atento ao tipo de dado utilizado para filtrar. No caso anterior, era um dado numérico (ID) e bastou escrevê-lo. Mas e se fosse um texto? Confira!

Observe os dados da tabela CLIENTE:

Tabela CLIENTE.

Você deseja o ID do cliente Ponto Quente, cujo valor é 110.

O comando seria o seguinte:

Consultando cliente.

Por que não voltou o ID do cliente, já que existe esse nome na tabela?
Alguns SGBD são *case sensitive* para os dados, ou seja, uma letra maiúscula é diferente da letra minúscula. Dessa forma, isso deve ser levado em consideração, ou a consulta poderá não retornar as linhas.

No caso do banco de dados, cada palavra do nome do cliente inicia com letra maiúscula. Portanto, o nome deve ser escrito dessa mesma forma no comando.

Consultando cliente.

Você pode estar se perguntando:

- Então eu tenho de saber como o nome está escrito no banco de dados?
- E se eu não souber?
- E se em uma linha estiver tudo maiúsculo e na outra estiver tudo minúsculo, de modo que o banco não tenha um padrão?

Nesse caso, você deve padronizar a consulta utilizando uma função que leve o valor existente no banco de dados para maiúsculo ou para minúsculo antes de fazer a comparação.

Veja o exemplo:

Consultando cliente.

Ao trabalharmos com datas, devemos colocá-las entre aspas simples, no formato dd/mm/aaaa, em que dd é o dia em dois dígitos, mm é o mês em dois dígitos e aaaa é o ano em quatro dígitos.

Acompanhe o seguinte exemplo: mostrar sobrenome e senha dos empregados admitidos em 03/03/2009.

Consultando empregado.

Consultando dados com várias condições

Vamos supor que você deseja mostrar os empregados que tenham sido contratados após primeiro de janeiro de 2010 e que tenham salário maior que 10000:

Consultando empregados.

Repare que a utilização do **AND** obriga que as duas condições sejam verdadeiras para que a linha retorne. Se tivéssemos utilizado OR, bastaria uma ser verdadeira.

Consultando empregados.

Operador IN

Permite comparar o valor da coluna com uma lista de valores e retorna verdadeiro se, em uma determinada linha, o valor da coluna for igual a um dos valores da lista.

Para listarmos os empregados que trabalham no departamento 10 ou no departamento 30, o comando seria o seguinte:

Consultando empregados.

Para consultar os empregados que **NÃO** trabalham nesses departamentos, basta acrescentar **NOT** ao comando:

Consultando empregados.

Operador BETWEEN

O operador **BETWEEN AND** permite verificar se o valor de um campo está contido em uma faixa de valores.

Por exemplo, para retornar o ID, ULT_NOME E SALÁRIO dos empregados com salários entre 8000 e 19500, inclusive, o comando seria o seguinte:

Consultando empregados.

Repare que retornam os empregados com salário de 8000 e de 19500, isso mostra que o comando BETWEEN cria um intervalo fechado, ou seja, os limites fazem parte dos valores aceitáveis. Para considerar a condição inversa, basta utilizarmos **NOT BETWEEN**.

Consultando empregados.

Operador LIKE

É utilizado para fazer casamento de padrão, ou seja, procurar um conjunto de caracteres que exista em uma cadeia de caracteres.

Para que essa operação de comparação seja eficiente, é necessário usar caracteres coringa, que, no caso do SQL, são dois:

- % – Representa uma quantidade arbitrária de caracteres (inclusive nenhum).
- _ – Indica a existência obrigatória de um caractere naquela posição.

Por exemplo, pode ser utilizado para saber o ID e ULT_NOME dos empregados com a letra E na segunda posição do sobrenome.

Consultando empregados.

Para os que não possuem, basta acrescentar **NOT** ao comando.

Consultando empregados.

Operador IS NULL

Determina se no campo existe valor ou se não existe (o campo é nulo).

Considere a tabela CLIENTE a seguir:

Tabela CLIENTE.

Se você deseja retornar todos os dados dos CLIENTES que não são atendidos por um vendedor, o comando seria o seguinte:

Consultando clientes.

Caso contrário, você utilizaria IS NOT NULL.

Consultando clientes.

Atividade 2

Observe a imagem a seguir e visualize o conteúdo de algumas colunas da tabela empregado em um SGBD PostgreSQL.

Tabela EMPREGADO.

Indique quais IDs de empregados seriam retornados caso fosse emitido comando a seguir:

LINGUAGEM SQL



Código SQL.

A 1.

B 1 e 2.

C 2 e 5.

D 1, 2 e 6.

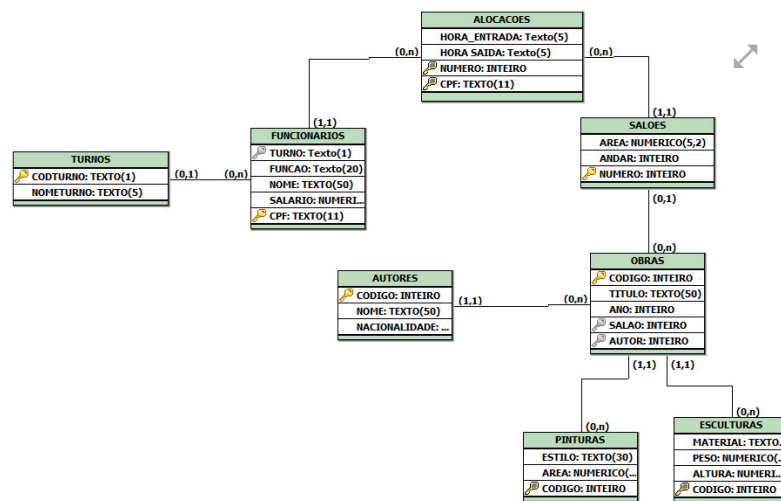
E 1, 2 e 5.

Parabéns! A alternativa E está correta.

O comando solicita o empregado cuja penúltima letra do ULT_NOME seja E.

Aplicação prática SELECT básico + WHERE

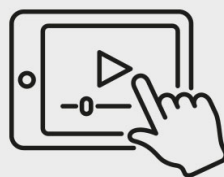
Veja alguns exemplos de uso dos comandos vistos até agora. Para isso, utilizaremos o banco de dados do museu cujo modelo lógico é o seguinte:



Modelo lógico do banco de dados MUSEU.

Neste vídeo, abordaremos a implementação do banco de dados do MUSEU no PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, siga os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta ao banco de dados do museu.
- Escreva os seguintes comandos de consulta:
 - Listar o conteúdo da tabela de autores.
 - Listar o nome e nacionalidade de todos os autores.
 - Listar CPF, nome, salário e salário anual dos funcionários.

- Listar o nome do autor concatenado com a sua nacionalidade.
- Listar o nome do autor concatenado com a sua nacionalidade com o alias de coluna nome e nacionalidade.
- Listar todos os dados dos autores com código maior do que 105.
- Listar todos os dados dos autores com código maior do que 105 e de nacionalidade francesa.
- Listar todos os dados dos autores franceses ou brasileiros.
- Listar todos os dados dos autores com código entre 104 e 107 inclusive.
- Listar todos os dados dos autores com Dali no nome.
- Listar todos os dados dos salões cuja área é nula.

Você pode baixar os comandos que resolvem as consultas em [sqlmod2np3](#).

Atividade 3

Explique a importância da cláusula WHERE em consultas SQL e forneça exemplos de situações em que seu uso é fundamental para a obtenção de resultados específicos.

Digite sua resposta aqui

Chave de resposta ▾

A cláusula **WHERE** é essencial em consultas SQL, pois permite filtrar os dados com base em condições específicas para obter resultados relevantes. As principais razões para o uso da cláusula WHERE incluem:

- **Filtragem de dados** – Permite filtrar os resultados da consulta com base em condições específicas.
- **Redução do volume de dados** – É essencial para consultas com grandes conjuntos de dados, reduzindo o volume de dados retornados e melhorando o desempenho da consulta.

- **Personalização dos resultados** – Permite personalizar os resultados da consulta de acordo com as necessidades do usuário.
- **Segurança dos dados** – Contribui para a segurança dos dados, restringindo o acesso apenas aos dados permitidos para um usuário específico.

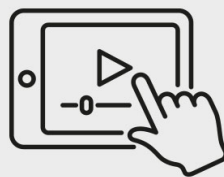
Agrupando e ordenando dados

Até aqui, todos os comandos retornavam toda linha da tabela que atendessem à condição da cláusula WHERE. Agora, veremos como gerar dados agregados utilizando funções de grupo, como eliminar linhas duplicadas e como ordenar a saída da consulta.

Nos exemplos, continuaremos utilizando o banco de dados da empresa. É recomendável que você execute os comandos de exemplo no PostgreSQL. Você pode encontrar no arquivo [sqltmod2nc4](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos os comandos SQL que permitem o agrupamento e a ordenação dos dados no PostgreSQL utilizando a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Eliminando valores duplicados (DISTINCT)

Análise o conteúdo da tabela EMPREGADO:

	id numeric (7)	ult_nome character varying (20)	prim_nome character varying (20)	cargo character varying (30)	salario numeric (7,2)	dt_admissao date	cpf character (11)	id_depto numeric (7)	id_gerente numeric (7)
1	1	Velasques	Carmen	Presidente	29500.00	2009-05-05	34567890125	10	null
2	2	Neves	Laurio	Diretor de Compras	19500.00	2009-03-03	23456789012	30	1
3	3	Nogueira	Ermene	Diretor de Vendas	18000.00	2010-04-07	34567890123	20	1
4	4	Queiroz	Mark	Gerente de Compras	8000.00	2010-11-11	12345432123	30	2
5	5	Rodrigues	Alberto	Vendedor	4000.00	2008-10-10	87965432123	20	3
6	6	Ugarde	Mariene	Vendedor	3500.00	2009-03-03	87654345678	20	3

Tabela EMPREGADO.

Observe que, na tabela empregados, Rodrigues e Ugarte possuem o mesmo cargo. Para consultar os diferentes cargos, considere o seguinte comando:

Consultando empregados.

Apesar de todos os cargos terem sido recuperados, os repetidos aparecem várias vezes. Para resolver esse tipo de caso, acrescenta-se a cláusula **DISTINCT** ao comando de **SELECT**, retornando apenas uma vez cada cargo.

Consultando empregados.

Observe o seguinte comando:

Consultando empregados.

Note que voltaram os dois vendedores, isso ocorreu porque a cláusula **DISTINCT** filtrou as linhas diferentes, não os valores da coluna. Como os dois vendedores possuem nomes diferentes, as linhas são distintas e, portanto, as duas retornam na busca.

Funções de grupo

Uma função de grupo atua em uma instância da tabela, ou seja, no conjunto de suas linhas. As funções agregam os dados a partir de todas as linhas da tabela ou de grupos em que as linhas possam ser enquadradas.

Na linguagem SQL, existem as funções para calcular a média (AVG) e a soma (SUM), determinar o maior e menor valor (MAX e MIN) e contar linha (COUNT).

	id numeric (7)	ult_nome character varying (20)	prim_nome character varying (20)	cargo character varying (30)	salario numeric (7,2)	dt_admissao date	cpf character (11)	id_depto numeric (7)	id_gerente numeric (7)
1		1 Velasques	Carmen	Presidente	29500.00	2009-05-05	34567890125	10	
2		2 Neves	Lauro	Diretor de Compras	19500.00	2009-03-03	23456789012	30	1
3		3 Nogueira	Ermene	Diretor de Vendas	18000.00	2010-04-07	34567890123	20	1
4		4 Queiroz	Mark	Gerente de Compras	8000.00	2010-11-11	12345432123	30	2
5		5 Rodrigues	Alberto	Vendedor	4000.00	2008-10-10	87965432123	20	3
6		6 Ugarte	Marlene	Vendedor	3500.00	2009-03-03	87654345678	20	3

Tabela EMPREGADO.

Para obter as agregações das funções para o salário do empregado, o comando seria o seguinte:

Consultando salários dos empregados.

Para listar o maior e o menor valor de salário de um empregado, os sobrenomes que aparecem como último e como primeiro na ordem

alfabética crescente e as datas do empregado contratado a mais tempo e a menos tempo, basta utilizar o seguinte comando:

Consultando com funções de grupo.

Criando grupos nas tabelas

Podemos agrupar os dados da tabela utilizando a cláusula **GROUP BY**. Por exemplo, para listar a quantidade de empregados, a média salarial e o valor da folha salarial de cada departamento, o comando seria o seguinte:

Consultando salários agrupados dos empregados.

Observe o seguinte:

- Na cláusula **GROUP BY**, você deve colocar a coluna cujos valores serão utilizados para criar os grupos, sendo que pertencem ao mesmo grupo todas as linhas que possuem valores iguais na coluna.
- Você também deve colocar na cláusula **SELECT** a coluna ou as colunas utilizadas para criar os grupos. Dessa forma, é possível saber a qual grupo pertence o valor gerado pela função de grupo.

A cláusula **GROUP BY** permite criar grupos definidos por mais de uma coluna. Isso significa que a agregação não será por uma coluna, mas por um conjunto de colunas.

Por exemplo, para recuperar a quantidade de empregados em cada cargo e em cada departamento o comando seria o seguinte:

Consultando salários agrupados dos empregados.

Filtrando dados agrupados

Ao se utilizar a cláusula **GROUP BY**, pode-se filtrar a seleção de dados antes de eles serem agrupados ou após o agrupamento.

O momento de realizar essa restrição depende de sua intenção: se deseja eliminar linhas da tabela antes de aplicar as funções de grupo, criando uma condição na cláusula **WHERE**; ou se deseja eliminar grupos que não atendam a uma determinada condição. Nesse caso, utiliza-se a cláusula **HAVING**.

Para filtrar as linhas antes de agrupar, por exemplo, contar a quantidade de empregados, o salário médio e a soma dos salários dos empregados do departamento 20, o comando é o seguinte:

Consultando salários dos empregados.

Além disso, podemos filtrar os grupos. Por exemplo, para retornar a quantidade de empregados, a média salarial e a soma dos salários para os departamentos que possuem mais de um empregado, o comando é o seguinte:

Consultando salários agrupados dos empregados.

Ordenando consultas

Até aqui, todos os comandos retornaram as linhas na ordem em que foram produzidas. No entanto, muitas vezes é necessário que elas sejam ordenadas. Para isso, o SQL utiliza a cláusula **ORDER BY**.

O resultado de uma consulta pode ser ordenado pelo valor de uma ou mais colunas de forma crescente, que é o padrão, ou decrescente, utilizando DESC.

Por exemplo, para retornar os dados dos empregados ordenados na ordem alfabética decrescente de ID_DEPTO e crescente de salário, o comando é o seguinte:

Consulta ordenada dos empregados.

Outra forma de comandar a ordenação é pela posição da coluna no resultado. Com isso, o comando anterior pode ser escrito da seguinte forma:

Consulta ordenada dos empregados.

É importante estar atento ao seguinte: a cláusula ORDER BY será sempre a última de um comando SQL.

Atividade 4

Na imagem a seguir, observe o conteúdo de algumas colunas da tabela empregado em um SGBD PostgreSQL.

Tabela empregado.

O retorno para o seguinte comando seria:

LINGUAGEM SQL



Código SQL.

- A Marlene
- B Ugarte
- C Velasques
- D Carmen
- E Neves

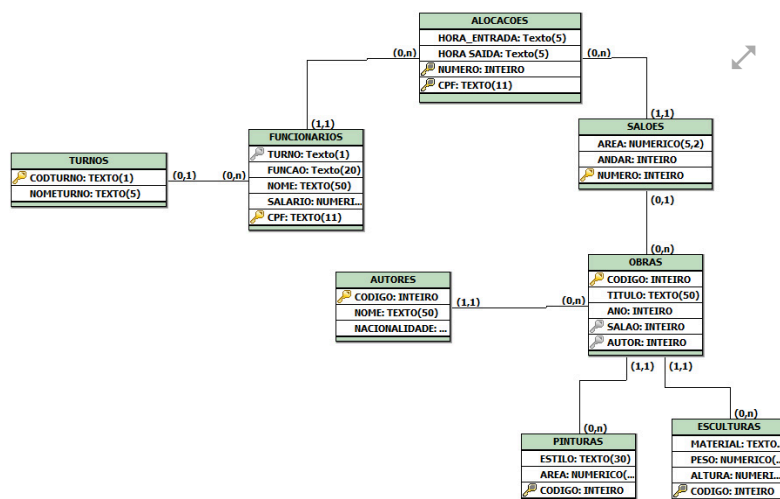
Parabéns! A alternativa A está correta.

O comando retorna o primeiro nome que aparece por último na ordem alfabética crescente.

Consulta à tabela EMPREGADO.

Aplicação prática de funções de agregação e GROUP BY + ORDER BY

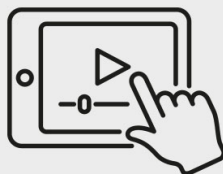
Veja alguns exemplos de uso dos comandos já aprendidos. Para isso, utilizaremos o banco de dados MUSEU, cujo modelo lógico é o seguinte:



Modelo lógico do banco de dados MUSEU.

Neste vídeo, abordaremos consultas SQL com funções de agregação, GROUP BY e ORDER BY.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, obedeça os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta do banco de dados do museu.
- Escreva os seguintes comandos de consulta:
 - Listar a área média com o alias área média, e número de salões com o alias total de salões.
 - Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas'.

- Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas' para os estilos diferentes de cubista.
- Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas' para os estilos diferentes de cubista e com pelo menos.
- Listar os diferentes estilos de pintura.
- Listar todos os dados dos autores em ordem descendente de nacionalidade.
- Listar todos os dados dos autores em ordem ascendente de nacionalidade e descendente de código.
- Repetir o comando anterior ordenando pela posição da coluna.

Você pode baixar os comandos que resolvem as consultas no arquivo [sqlmod2np5](#).

Atividade 5

Explique a importância das funções de grupo (como COUNT, SUM, AVG, MAX e MIN) em consultas SQL e como são essenciais para a análise de dados.

Digite sua resposta aqui

Chave de resposta ▾

As funções de grupo desempenham um papel fundamental na análise de dados em consultas SQL, permitindo resumir e agregar informações de maneira eficiente. Veja a importância dessas funções:

- **Agregação de dados** – as funções de grupo permitem agregar dados em grupos com base em uma ou mais colunas, fornecendo informações resumidas sobre conjuntos de dados grandes.
- **Análise estatística** – as funções de grupo, como AVG (média), MAX (valor máximo) e MIN (valor mínimo),

permitem realizar análises estatísticas rápidas e eficientes em conjuntos de dados.

- **Identificação de tendências** – ao usar funções de grupo em conjunto com cláusulas de filtragem e ordenação, podemos identificar tendências e padrões nos dados.
- **Tomada de decisão** – as informações resumidas fornecidas pelas funções de grupo são essenciais para as tomadas de decisão informadas.



3 - Comandos SELECT com duas ou mais tabelas

Ao final deste módulo, você será capaz de aplicar consultas SQL para mais de uma tabela.

Ligando os pontos

Você trabalha na equipe de desenvolvimento de sistemas da TechStore, uma empresa do setor de comércio eletrônico, especializada na venda de produtos eletrônicos, como smartphones, laptops, tablets e acessórios relacionados. Possui uma gama de produtos de diversas marcas renomadas e busca proporcionar aos clientes uma experiência de compra conveniente e eficiente.

O sistema necessita da elaboração de três novas consultas assim descritas:

1. **Consulta para listar os produtos mais vendidos** – identificar os produtos mais populares vendidos pela TechStore.

2. **Consulta para calcular o total de vendas por mês** – calcular o total de vendas realizadas pela TechStore em cada mês.
3. **Consulta para identificar clientes mais frequentes** – identificar os clientes que realizaram o maior número de compras na TechStore.

Após a leitura do case, é hora de aplicar seus conhecimentos! Vamos ligar esses pontos?

Questão 1

Como pode ser realizada a consulta para identificar os clientes mais frequentes na TechStore?

- A Usando a função AVG para calcular a média de compras de cada cliente ao longo do tempo.
- B Utilizando a função COUNT para contar o número total de clientes cadastrados na TechStore.
- C Escrevendo uma consulta com a cláusula WHERE para filtrar clientes com mais de uma compra.
- D Criando uma subconsulta correlata para contar o número de pedidos feitos por cada cliente.
- E Aplicando uma junção externa para combinar as tabelas de clientes e pedidos e agrupando os resultados por cliente.

Parabéns! A alternativa D está correta.

Para identificar os clientes mais frequentes na TechStore, é necessário contar o número de pedidos feitos por cada cliente. Uma subconsulta correlata é a abordagem adequada para isso, pois pode ser utilizada para calcular o número de pedidos associados a cada cliente. A subconsulta correlata permite relacionar a tabela de clientes com a tabela de pedidos e contar quantos pedidos estão associados a cada cliente individualmente. Dessa forma, podemos identificar os clientes mais frequentes com base na contagem de pedidos realizados.

Questão 2

Qual é a melhor abordagem para realizar a consulta que calcula o total de vendas por mês na TechStore?

- A Utilizar a função COUNT para contar o número total de vendas e agrupar os resultados por mês.
- B Criar uma subconsulta correlata para obter o total de vendas em cada mês.
- C Escrever uma consulta com várias cláusulas WHERE para filtrar as vendas por mês.
- D Aplicar uma junção externa entre as tabelas de vendas e datas para combinar os dados e agrupar os resultados por mês.
- E Utilizar a função SUM para somar o valor total das vendas e agrupar os resultados por mês.

Parabéns! A alternativa D está correta.

Para calcular o total de vendas por mês na TechStore, é necessário combinar as informações da tabela de vendas com as datas dos pedidos. Uma junção externa entre essas tabelas permite unir os dados corretamente e agrupá-los por mês, o que facilita o cálculo do total de vendas em cada período. Isso garante que todas as vendas sejam incluídas na consulta e que os resultados sejam corretamente agregados por mês.

Questão 3

Descreva uma estratégia para realizar a consulta que visa identificar os produtos mais vendidos na TechStore. Explique como você empregaria subconsultas ou junções para alcançar esse objetivo e discuta as vantagens de sua abordagem.

Digite sua resposta aqui

Uma abordagem eficaz para identificar os produtos mais vendidos na TechStore seria utilizar uma subconsulta correlata para contar o número de unidades vendidas de cada produto na tabela de pedidos e, em seguida, incorporá-la à consulta principal para selecionar os produtos com o maior número de vendas. Uma outra abordagem seria utilizar uma junção entre as tabelas de produtos e detalhes do pedido para combinar os dados relevantes, agrupar e contar o número de vendas de cada produto, garantindo que apenas os produtos vendidos sejam considerados na análise, o que permite uma consulta mais eficiente e direta.

Subconsultas

Muitas vezes, é necessário em uma consulta filtrar as linhas por valores que não existem no banco de dados. Por exemplo, para selecionar os empregados cujo salário seja maior que o salário médio de todos os empregados, existem as subconsultas. Confira!

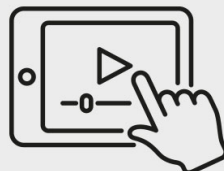
Você pode encontrar no arquivo [sqlmod3nc1](#) os comandos utilizados nos exemplos.



Subconsultas aninhadas e correlatas

Neste vídeo, abordaremos os comandos SQL para subconsultas aninhadas e correlatas no PostgreSQL utilizando a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Subconsultas aninhadas

São consultas secundárias em relação a outras consultas e são utilizadas para gerar valores intermediários necessários para executar a consulta principal.

Considere que você deseja recuperar todos os dados dos empregados cujo salário seja maior que o salário médio da empresa. Nesse caso, o comando seria:



```

1 SELECT *
2 FROM EMPREGADO
3 WHERE SALARIO >= ( SELECT AVG(SALARIO) FROM EMPREGADO)

```

	id (PK) numeric (7)	ult_nome character varying (20)	prim_nome character varying (20)	cargo character varying (30)	salario numeric (7,2)	dt_admissao date	cpf character (11)	id_depto numeric (7)
1	1	Velasques	Carmen	Presidente	29500.00	2009-05-05	34567890125	10
2	2	Neves	Lauro	Diretor de Compras	19500.00	2009-03-03	23456789012	30
3	3	Nogueira	Ernane	Diretor de Vendas	18000.00	2010-04-07	34567890123	20

Exemplo de subconsulta.

Observe que existe um comando **SELECT** (consulta inicial) e um outro comando **SELECT** dentro deste primeiro (interno), o que caracteriza a subconsulta aninhada.

Subconsulta simples

São aquelas que retornam apenas uma linha, como acontece, por exemplo, ao se utilizar uma função de grupo sem cláusula **GROUP BY** como no caso da subconsulta como a que calculou a média salarial.

Nesse tipo de subconsulta, podem ser utilizados os seguintes operadores na comparação: **=**, **<>**, **>**, **<**, **>=** e **<=**.

Subconsultas multilinhas

São aquelas que retornam mais de uma linha, o que requer a utilização da comparação **IN** ou **NOT IN**, pois somente esses operadores conseguem lidar um conjunto de valores. Veja o exemplo: a seguir.

A tabela **DEPARTAMENTOS** tem o seguinte conteúdo:

Tabela **DEPARTAMENTO**.

Temos dois departamentos da região 1 e um da região 2. Se quisermos listar o **ID**, **ULT_NOME** e **ID_DEPTO** dos empregados do departamento da região 1, como há dois departamentos, não é possível utilizar **=**, então utiliza-se **IN**:

Exemplo de subconsulta.

Subconsulta correlata

Além das subconsultas aninhadas, existe outro tipo de consulta denominada correlata. Para executá-la, é necessário utilizar valores da consulta principal, por isso deve ser executada uma vez para cada linha da consulta externa.

Por exemplo, considere que você deseja retornar o ID, ULT_NOME, SALÁRIO, ID_DEPTO dos empregados que ganham igual ou mais que o salário médio de seu departamento. Para resolver isso é necessário que:

1. A subconsulta acesse na consulta principal o departamento do empregado cuja linha está sendo processada naquele momento.
2. Calcule a média do valor para aquele departamento.
3. Retorne a média para consulta principal.
4. A consulta principal compare a média com o salário do empregado da linha que está sendo processada.
5. Retorne os dados do empregado se o valor segurado for menor que a média.

Exemplo de subconsulta correlata.

Observe no comando que a tabela externa foi apelidada de E a interna de I e no WHERE foi utilizado o valor da externa para filtrar as linhas da subconsulta.

Exemplo de subconsulta correlata.

Atividade 1

Observe a imagem a seguir e visualize o conteúdo de algumas colunas da tabela EMPREGADO em SGBD PostgreSQL:

Tabela EMPREGADO.

Indique a alternativa que apresenta os ID de empregados resultantes do seguinte comando:

LINGUAGEM SQL



Código SQL.

A 2 e 4.

B 1.

C 3 e 5.

D 1, 2, 5 e 6.

E 2, 5 e 6.

Parabéns! A alternativa A está correta.

O comando solicita ao empregado cujo ID_DEPTO seja maior que o ID_DEPTO do empregado cujo cargo tenha a palavra VENDAS no final, que corresponde ao empregado Nogueira do departamento 20; portanto, voltam os empregados do departamento 30.

Exemplo de subconsulta.

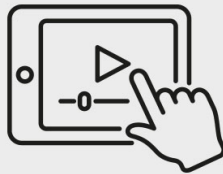
Operadores de conjunto

Algumas vezes, é necessário realizar uma junção vertical, que consiste em retornar dados de mais de uma tabela em uma mesma coluna. Isso pode ser feito utilizando os operadores de conjunto UNION, INTERSECT e EXCEPT. Confira!

Você pode encontrar no arquivo [sqlmod3nc2](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos os comandos SQL utilizando operadores de conjunto, tal como UNION, INTERSECT e EXCEPT, no PostgreSQL utilizando a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Utilizando operadores de conjunto

Toda tabela pode ser considerada um conjunto de valores, e os operadores de conjunto permitem realizar as seguintes operações:



União

O operador UNION retorna todas as linhas do primeiro conjunto acrescidas de todas as linhas do segundo conjunto, eliminando as duplicadas.



Interseção

O operador INTERSECT retorna as linhas comuns aos dois conjuntos.



Diferença

O operador EXCEPT retorna as linhas que existem no primeiro conjunto e não existem no segundo.

Para gerar os conjuntos a serem operados, são utilizadas subconsultas para gerar tabelas em memória. Para que o comando funcione, é

necessário que as tabelas geradas sejam compatíveis.

Duas ou mais tabelas são compatíveis para fazer operações de conjunto se possuem o mesmo esquema, ou seja, o mesmo número de colunas, como mesmo tipo na mesma ordem. Observe as tabelas a seguir:

Tabela Empregados		
ID	SALÁRIO	NOME
101	5000,00	Carlos Antunes
102	1100,00	Pedro Leitão
104	1500,00	Marco Aurélio
Tabela Produtos		
CÓDIGO	PREÇO	TEXTO
11	75,00	Fechadura
12	20,00	Cadeado
14	5,00	Aldraba
Tabela Animais		
ESPÉCIE	PESO MÉDIO	TEMPO VIDA
Cavalo	400	35
Gato	3	20
Cachorro	25	12

Compatibilidade entre tabelas.

As tabelas EMPREGADOS e PRODUTOS são compatíveis já que possuem três colunas, sendo a primeira e a segunda numéricas e a terceira, alfanumérica. Já a tabela ANIMAIS não é compatível com as outras duas, apesar de também possuir três colunas, sendo duas numéricas e uma alfanumérica, elas não se encontram na mesma ordem, pois a alfanumérica é a primeira e as numéricas correspondem às duas últimas.

Operador UNION

Realiza a operação de união, ou seja, retorna todas as linhas do primeiro conjunto acrescidas de todas as linhas do segundo conjunto, eliminando as duplicadas.

Para obter, por exemplo, os IDs dos empregados que trabalham nos departamentos 20 ou 30 ou que possuem o cargo de vendedor, como ficaria o comando utilizando operadores de conjunto?

Observe os dados da tabela a seguir:

Tabela DEPARTAMENTO.

Note que os empregados de ID 2 a 6 são dos departamentos 20 ou 30 (destacados em azul), e os empregados de ID 5 e 6 são vendedores (destacados em vermelho). Você precisa dos dados que estão em um conjunto ou no outro, portanto, os empregados de ID 2, 3, 4, 5 e 6. Note ainda que os empregados de ID 5 e 6 aparecem nos dois conjuntos. O comando que resolve o problema é o seguinte:

Exemplo de uso do comando UNION.

Note que:

1. Retornaram os ID 2,3 e 4 apesar de não serem vendedores.
2. Os empregados de ID 5 e 6 retornaram apenas uma vez, apesar de estarem nos dois conjuntos.

Existe uma variação no SQL do comando de união que retorna os duplicados, trata-se do comando UNION ALL.

Exemplo de uso do comando UNION ALL.

Observe que agora os IDs 5 e 6 aparecem duas vezes.

Operador INTERSECT

Para obter o ID dos empregados que trabalham nos departamentos 20 ou 30 e que possuem o cargo de vendedor, ou seja, os IDs que aparecem nos dois conjuntos, utiliza-se o comando INTERSECT para obter a interseção dos conjuntos.

Exemplo de uso do comando INTERSECT.

Operador EXCEPT

As operações de união e interseção são **comutativas**, ou seja, tanto faz a primeira subconsulta ser pelo modelo e a segunda pela cor, ou o inverso, pois o resultado será igual.

Isso não acontece na diferença (EXCEPT). Nesse caso, ao inverter a ordem, o resultado mudará.

Para retornar o ID dos empregados que trabalham nos departamentos 20 ou 30 e que **não** possuem o cargo de vendedor, ou seja, estão no primeiro conjunto e não estão no segundo, o comando é o seguinte:

Exemplo de uso do comando EXCEPT.

Já para retornar os vendedores que não são dos departamentos 20 e 30, o comando é o seguinte:

Exemplo de uso do comando EXCEPT.

Atividade 2

Observe a imagem a seguir e visualize o conteúdo de algumas colunas da tabela EMPREGADO em SGBD PostgreSQL:

Tabela EMPREGADO.

Consulte o comando a seguir e indique a alternativa com os IDs de empregados que retornariam:

LINGUAGEM SQL



Código SQL.

A 2, 4, 5 e 6.

B 2, 3, 4, 5 e 6.

C 1, 3, 5, e 6.

D 1, 3, 4, 5 e 6.

E 1, 2, 3, 4, 5 e 6.

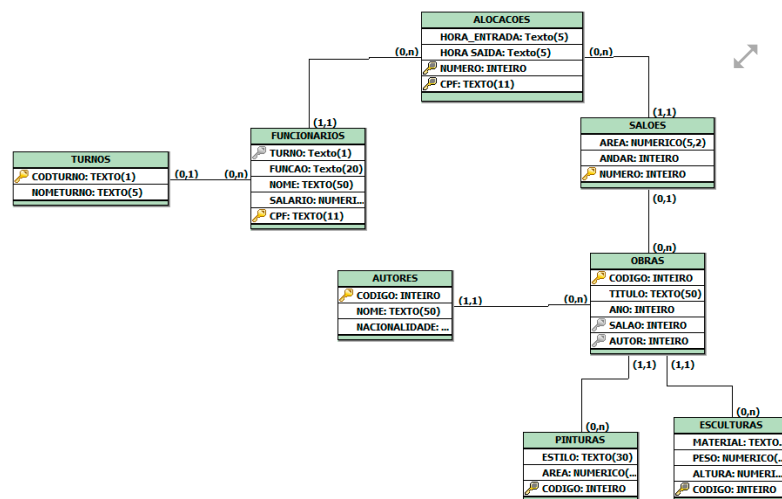
Parabéns! A alternativa D está correta.

No comando emitido, o primeiro conjunto é formado pelos empregados de ID 3, 4 e 5, já o segundo conjunto é composto pelos empregados de ID 1, 3, 5 e 6. Como foi solicitada a união, o resultado é composto pelos empregados de ID 1, 3, 4, 5 e 6.

Comando UNION.

Aplicação prática de subconsultas e operações sobre conjuntos

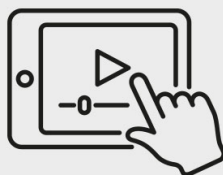
Veja alguns exemplos de uso dos comandos já aprendidos. Para isso, utilizaremos o banco de dados do museu, cujo modelo lógico é o seguinte:



Modelo lógico do banco de dados MUSEU.

Neste vídeo, abordaremos as consultas SQL com aplicação prática de subconsultas e operações sobre conjuntos, incluindo UNION, INTERSECT e EXCEPT.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, respeite os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta ao banco de dados do museu.
- Escreva os seguintes comandos de consulta:
 - Listar todos os dados de funcionários cujo salário seja maior do que o salário médio de todos os funcionários.
 - Listar todos os dados dos salões que possuem obras expostas.
 - Listar todos os dados dos salões que não possuem obras expostas no museu.
 - Listar os dados dos autores que não possuem obras expostas no museu.
 - Listar todos os dados dos salões cujo área seja maior do que a área média dos salões do mesmo andar.
 - Listar todos os dados do autores que possuem obras utilizando teste de existência.

- Listar todos os dados do autores que não possuem obras utilizando teste de existência.
- Listar código dos autores que não possuem obras expostas no museu utilizando operadores de conjunto.
- Listar código dos autores que possuem obras expostas no museu utilizando operadores de conjunto.
- Listar o código dos autores que possuem apenas pinturas no museu utilizando operadores de conjunto.
- Listar o código dos autores que possuem apenas esculturas no museu utilizando operadores de conjunto.
- Listar o código dos artistas que possuem ou apenas esculturas ou apenas pinturas utilizando operadores de conjunto.

Você pode baixar os comandos que resolvem as consultas no arquivo [sqlmod3np3](#).

Atividade 3

Explique a diferença entre subconsulta correlata e subconsulta aninhada em um contexto de consulta SQL. Destaque as características distintivas de cada tipo de subconsulta e discuta suas respectivas aplicações.

Digite sua resposta aqui

Chave de resposta ▾

As subconsultas correlatas e aninhadas possuem diferenças fundamentais em suas abordagens e aplicações em SQL.

Uma subconsulta correlata é uma subconsulta executada repetidamente para cada linha retornada pela consulta externa, e faz referência a uma ou mais colunas da consulta externa. Isso permite que a subconsulta seja avaliada dinamicamente para cada linha, com base nos valores específicos das colunas daquela linha. Esse tipo de consulta é frequentemente usado para filtrar ou agregar

dados com base em condições específicas relacionadas a cada linha da consulta principal.

Por outro lado, uma subconsulta aninhada é uma subconsulta é executada independentemente da consulta externa e produz um conjunto de resultados utilizado na consulta externa. Esse tipo de consulta é tratado como uma tabela temporária, e seu resultado pode ser referenciado na cláusula FROM, JOIN ou WHERE da consulta principal. É necessária para realizar operações complexas ou computações antes de unir os resultados com outras tabelas ou filtrá-los de alguma forma.

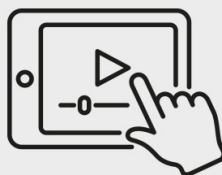
Junções internas e externas

Para retornar dados de uma tabela, como o nome do departamento armazenado na tabela DEPARTAMENTO e o nome da região armazenado na tabela REGIÃO, é necessário realizar uma junção.

Você pode encontrar no arquivo [sqlmod3nc4](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos os comandos SQL, utilizando junções internas e externas no PostgreSQL, com a ferramenta pgAdmin.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Junção interior

Voltemos à discussão sobre retornar dados das tabelas DEPARTAMENTO e REGIÃO. Ao observar o esquema da tabela DEPARTAMENTO, nota-se a existência de uma chave estrangeira (ID_REGIAO) que referência a tabela REGIÃO, o que permite saber a qual região o departamento pertence.

Tabelas DEPARTAMENTO e REGIÃO.

Para obter dados dos departamentos e de suas regiões, por exemplo, utiliza-se a FK ID_REGIAO para conectar as linhas das duas tabelas, realizando uma junção.

Como regra geral, podemos definir que, para realizar uma junção, também conhecida como JOIN, determina-se as colunas em comum nas duas tabelas (PK e FK) e utiliza-se seus valores para determinar as linhas que serão retornadas.

A sintaxe padrão definida pela ANSI para esse tipo de junção é a seguinte:

LINGUAGEM SQL



Código SQL.

Nessa sintaxe, a condição de junção é uma expressão de comparação entre as colunas das duas tabelas envolvidas, normalmente a chave primária de uma com a chave estrangeira da outra.

Vamos comandar a junção das tabelas DEPARTAMENTO e REGIÃO e retornar todas as colunas.

Exemplo de uso do comando JOIN.

Observe um detalhe no comando: como as duas tabelas possuem uma coluna ID_REGIAO, é preciso realizar a qualificação, ou seja, dizer a qual tabela pertence a coluna ID_REGIAO que estamos referenciando, senão acontecerá um erro de coluna ambígua.

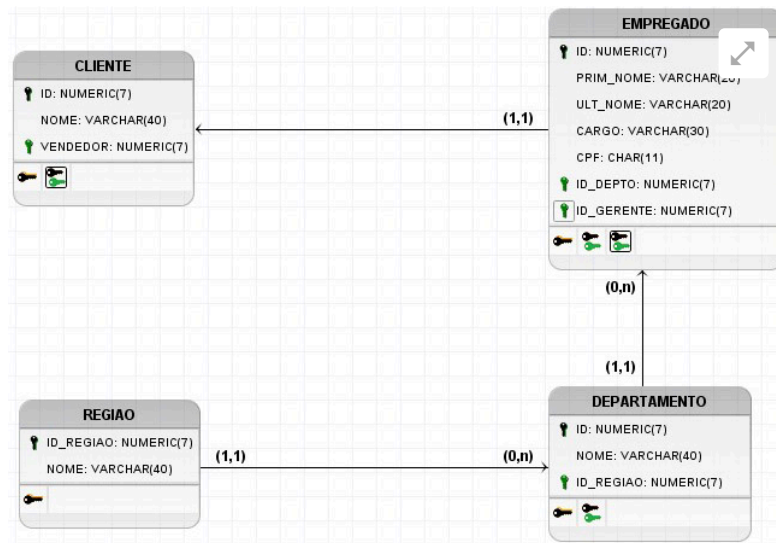
Exemplo de uso do comando JOIN.

Para facilitar a qualificação, podemos atribuir um alias à tabela, normalmente uma letra para substituir seu nome. Nesse caso, o comando ficaria da seguinte forma:

Exemplo de uso do comando JOIN.

Junção de mais de duas tabelas

Para realizar a junção de mais de duas tabelas, basta acrescentarmos uma tabela com uma nova cláusula de junção. Observando as tabelas de exemplo a seguir, a primeira coisa a fazer é identificar quais podem se juntar. Analise o modelo do banco apresentado:



Modelo lógico.

Tem-se uma FK de EMPREGADO para DEPARTAMENTO e uma FK de DEPARTAMENTO para REGIAO. Dessa forma, deve-se comandar a junção de EMPREGADO com DEPARTAMENTO, gerando uma tabela intermediária em memória, e depois juntar este resultado com REGIAO.

Exemplo de uso do comando JOIN.

Autojunção

É uma junção da tabela com ela mesma. Na tabela EMPREGADO, cada empregado está subordinado a outro. A coluna ID_GERENTE indica o código do gerente do empregado.

Exemplo de autojunção.

Para mostrar uma lista de todos os gerentes, cada um com seus subordinados, utiliza-se o comando INNER JOIN. Confira na imagem a seguir:

```

SELECT G.ID, G.ULT_NOME, G.CARGO, S.ID, S.ULT_NOME, S.CARGO
FROM EMPREGADO G
INNER JOIN EMPREGADO S ON S.ID_GERENTE = G.ID
  
```

id [PK] numeric (7)	ult_nome character varying (20)	cargo character varying (30)	id numeric (7)	ult_nome character varying (20)	cargo character varying (30)
1	Velasques	Presidente	2	Neves	Diretor de Compras
1	Velasques	Presidente	3	Nogueira	Diretor de Vendas
2	Neves	Diretor de Compras	4	Queiroz	Gerente de Compras
3	Nogueira	Diretor de Vendas	5	Rodrigues	Vendedor
3	Nogueira	Diretor de Vendas	6	Ugarte	Vendedor

Exemplo de uso do comando INNER JOIN.

Nesse caso, é obrigatório usar um apelido de tabela para distinguir as duas cópias da tabela relacionadas: "G", no exemplo, representa uma linha da tabela EMPREGADO, enquanto "Gerente" e "S" representam uma linha de um subordinado.

Junção exterior

Esse tipo de junção retorna as linhas que estão relacionadas, como no INNER JOIN, e as não relacionadas de uma ou mais tabelas. Considere a situação dos empregados e clientes mostrada na imagem a seguir:

Entendendo a junção exterior.

Para realizar a junção interior, o comando seria:

Exemplo de junção interior.

Note que no resultado voltam apenas os empregados de ID 5 e 6, pois estão relacionados como clientes. Os demais não aparecem no resultado (ID 1, 2, 3 e 4). Mas e se desejássemos retornar todos os empregados e, no caso dos vendedores da empresa, quiséssemos também os dados dos seus clientes? É necessário comandar uma junção exterior.

Nesse caso, substitui-se a expressão **INNER** pela expressão **LEFT** ou **RIGHT** caso a tabela a ser retornada esteja do lado esquerdo ou direito do comando.

Como EMPREGADO está à esquerda, o comando é o seguinte:

Exemplo de junção exterior.

Note que as colunas de clientes ficam nulas nas linhas que correspondem aos empregados que não se relacionam com clientes. E, se o comando fosse de RIGHT JOIN, o que aconteceria? Confira!

Exemplo de junção exterior.

O comando retornaria todos os clientes, e, para os que não se relacionam com vendedores (Casa Desconto), as colunas de empregado seriam nulas. Uma terceira forma de fazer a junção exterior é com o **FULL JOIN**.

Exemplo de junção exterior.

O comando **FULL JOIN** retorna todos os dados relacionados e os não relacionados das duas tabelas, preenchendo com nulo as colunas da outra tabela para as linhas não relacionadas.

Atividade 4

Observe a imagem a seguir e visualize o conteúdo de algumas colunas da tabela EMPREGADO em SGBD PostgreSQL:

Consulta à tabela EMPREGADO.

Consulte o seguinte comando e indique a alternativa que apresenta os GID de empregados que retornariam:

LINGUAGEM SQL



Código SQL.

- A 1, 2 e 3.
- B 2, 3, 4, 5 e 6.
- C 3, 5, e 6.
- D 3, 4, 5 e 6.
- E 4, 5 e 6.

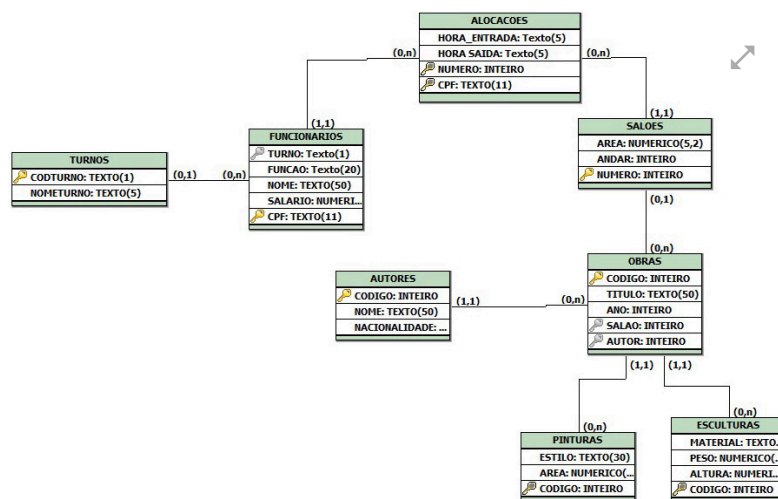
Parabéns! A alternativa E está correta.

É executado um comando de junção exterior no modelo de autojunção, o que faz com que mesmo os empregados que não gerenciam ninguém apareçam como gerentes, ou seja, nas colunas G. Como na cláusula WHERE eliminamos as linhas que estão com as colunas S.ID, restam apenas os empregados que não são gerentes, ou seja, 4, 5 e 6.

Consulta à tabela EMPREGADO utilizando junção externa.

Aplicação prática de junções

Veja alguns exemplos de uso dos comandos já aprendidos. Para isso, utilizaremos o banco de dados MUSEU, cujo modelo lógico é o seguinte:



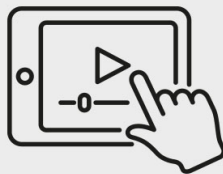
Modelo lógico do banco de dados MUSEU.



Aplicação prática de junções

Neste vídeo, abordaremos os comandos de junção com SQL no PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro na prática

Para executar a atividade, respeite os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta ao banco de dados do museu.
- Escreva os seguintes comandos de consulta:
 - Listar todos os dados dos autores e de suas obras.
 - Listar o código o título e o nome do autor das obras de 1965 a 1975 que estão no salão 36.
 - Listar o nome e a nacionalidade dos autores que possuem obras expostas, resolver por junção.
 - Listar o nome e a nacionalidade dos autores que não possuem obras expostas, resolver por junção.
 - Listar o código e o título das obras do autor Pablo Picasso que se encontram no terceiro andar do museu.
 - Listar o nome e a nacionalidade dos autores, o título da obra e o estilo da pintura.
 - Listar todos os dados dos salões que não possuem obras expostas.
 - Listar o código e o título das obras impressionistas ou cujo material de fabricação é argila, resolva por junção.
 - Gerar uma relação que associa pares de nomes de seguranças diferentes que cuidam dos mesmos salões nos mesmos períodos (mesmos horários de entrada e saída).

Você pode baixar os comandos que resolvem as consultas no arquivo [sqlmod3np5](#).

Atividade 5

Explique quando e por que você usaria cada tipo de junção (INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL JOIN) em uma consulta SQL. Destaque as situações em que cada tipo de junção é mais apropriado e discuta os resultados esperados em cada caso.

Digite sua resposta aqui

Chave de resposta ▾

Cada tipo de junção (INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL JOIN) tem sua própria aplicação e é útil em diferentes situações ao escrever consultas SQL:

INNER JOIN – É usado para retornar apenas os registros que possuem correspondência em ambas as tabelas envolvidas na junção. Combina linhas de duas ou mais tabelas com base em uma condição de igualdade especificada, excluindo registros que não têm correspondência.

OUTER JOIN (LEFT, RIGHT ou FULL) – São usados para incluir registros de uma tabela mesmo que não haja correspondência com a outra tabela na junção. Incluem registros que podem não ter correspondência com base na condição de junção especificada.



4 - Linguagem procedural de banco de dados

Ao final deste módulo, você será capaz de programar utilizando PL/pgSQL.

Ligando os pontos

Você foi contratado como desenvolvedor na HealthyFit, uma startup que oferece serviços de treinamento físico personalizado e acompanhamento nutricional on-line. Para gerenciar clientes, treinadores e planos de treinamento, a HealthyFit decidiu implementar um banco de dados relacional.

O banco de dados da HealthyFit possui as seguintes tabelas principais:

- **CLIENTES** – Armazena informações sobre os clientes da HealthyFit, incluindo ID do cliente, nome, idade, peso, altura e informações de contato.
- **TREINADORES** – Registra os treinadores da HealthyFit, contendo informações como ID do treinador, nome, especialização e informações de contato.
- **PLANOS** – Registra os planos de treinamento personalizado oferecidos pela HealthyFit, incluindo ID do plano, descrição, duração e preço.
- **REGISTRO_DE_TREINAMENTO** – Registra as sessões de treinamento realizadas pelos clientes, contendo informações como ID do cliente, ID do treinador, data da sessão, duração e observações.

Com base nesse contexto, a HealthyFit solicita a implementação dos seguintes elementos:

- **Função** – Criar uma função chamada "CalcularIMC" que calcule o Índice de Massa Corporal (IMC) de um cliente com base no peso e altura fornecidos.
- **Procedimento armazenado** – Criar um procedimento armazenado chamado "CriarPlanoTreinamento" que permita aos treinadores criarem um novo plano de treinamento personalizado para um cliente específico, inserindo-o na tabela de planos.
- **Gatilho** – Implementar um gatilho que, sempre que uma nova sessão de treinamento for registrada na tabela "Registro_de_Treinamento", atualize automaticamente o registro de histórico de treinamento do cliente na tabela CLIENTES, adicionando a data da última sessão e o número total de sessões realizadas.

Após a leitura do case, é hora de aplicar seus conhecimentos! Vamos ligar esses pontos?

Questão 1

Qual é a finalidade da função "CalcularIMC" no contexto da empresa HealthyFit?

- A Validar a autenticação de clientes e treinadores no sistema.
- B Calcular o índice de massa corporal (IMC) de um cliente com base em seu peso e altura.
- C Gerar relatórios financeiros sobre os planos de treinamento oferecidos pela empresa.
- D Atualizar o status dos clientes com base na conclusão de seus planos de treinamento.
- E Enviar notificações aos treinadores sobre as sessões de treinamento agendadas.

Parabéns! A alternativa B está correta.

A função "CalcularIMC" tem como objetivo principal calcular o índice de massa corporal (IMC) de um cliente com base nas informações de peso e altura fornecidas. O IMC avalia o peso ideal de acordo com a altura, sendo uma informação importante no contexto de um serviço de treinamento físico e acompanhamento nutricional ofertado pela HealthyFit.

Questão 2

Como um procedimento armazenado como o "CriarPlanoTreinamento" pode melhorar o trabalho dos treinadores na empresa HealthyFit?

- A Automatizando o processo de avaliação de desempenho dos treinadores.
- B Facilitando a comunicação entre treinadores e clientes.

- C Padronizando os métodos de treinamento utilizados pelos treinadores.
- D Permitindo uma criação rápida e eficiente de planos de treinamento personalizados para os clientes.
- E Reduzindo a carga de trabalho administrativo dos treinadores.

Parabéns! A alternativa D está correta.

Um procedimento armazenado como o "CriarPlanoTreinamento" pode melhorar significativamente o trabalho dos treinadores ao possibilitar a criação rápida e eficiente de planos de treinamento personalizados para os clientes. Isso permite que os treinadores atendam às necessidades específicas de cada cliente de forma mais eficaz e rápida, economizando tempo e esforço na elaboração manual de planos de treinamento.

Questão 3

Descreva como a implementação de um gatilho pode beneficiar a empresa HealthyFit e seus processos operacionais. Além disso, discuta um cenário específico em que um gatilho poderia ser útil e como ele poderia ser configurado para melhorar a eficiência ou a integridade dos dados no contexto da empresa.

Digite sua resposta aqui

Chave de resposta ▾

A implementação de gatilhos pode trazer vários benefícios para a empresa HealthyFit, incluindo automação de processos e garantia da integridade dos dados. Um cenário útil seria a atualização automática do registro de histórico de treinamento de um cliente sempre que uma nova sessão de treinamento for registrada na tabela `REGISTRO_DE_TREINAMENTO`. Isso manteria um registro preciso e atualizado das atividades de treinamento de cada

cliente, sem depender de atualizações manuais. Além disso, isso melhoraria a eficiência operacional, eliminando atualizações manuais, garantindo registros precisos e atualizados, e mantendo a integridade dos dados na HealthyFit.

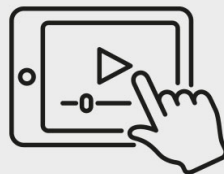
Linguagem procedural – PL/pgSQL

Procedural Language/PostgreSQL (PL/pgSQL) é uma linguagem de programação procedural suportada pelo PostgreSQL. Trata-se de uma linguagem de programação completa, incluindo a capacidade de usar *loops* e outras estruturas de controle.

Você pode encontrar no arquivo [sqlmod4nc1](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos a estrutura da linguagem de programação procedural PL/pgSQL suportada pelo PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Estrutura do PL/pgSQL

PL/pgSQL é uma linguagem estruturada em blocos, e cada instrução dentro de um bloco é terminada por um ponto e vírgula. A sintaxe básica é a seguinte:

LINGUAGEM SQL



Código SQL.

A seguir, conheça a estrutura do bloco com mais detalhes:

- Cada bloco tem duas seções: declaração e corpo. A seção de declaração é opcional, enquanto a seção do corpo é obrigatória. Um bloco é finalizado com um ponto e vírgula (;) após o END.
- Um bloco pode ter um rótulo opcional localizado no início. O rótulo do bloco é utilizado para especificar o bloco na instrução do corpo ou para qualificar os nomes das variáveis declaradas.
- A seção de declaração é onde você declara todas as variáveis usadas na seção do corpo. Cada instrução na seção de declaração termina com um ponto e vírgula (;).
- A seção do corpo é onde se coloca o código. Cada instrução na seção do corpo também é encerrada com um ponto e vírgula (;).

Declaração de variáveis

Todas as variáveis utilizadas em um bloco devem ser declaradas na seção de declarações do bloco. As variáveis da linguagem PL/pgSQL podem possuir qualquer tipo de dado da linguagem SQL, como integer, varchar e char. Veja os exemplos a seguir.

LINGUAGEM SQL



Código SQL.

A sintaxe geral para declaração de variáveis é a seguinte:

LINGUAGEM SQL



Código SQL.

A cláusula **DEFAULT**, se for fornecida, especifica o valor inicial atribuído à variável quando o processamento entra no bloco. Se a cláusula **DEFAULT** não for fornecida, então a variável é inicializada com o valor nulo do SQL.

Instruções básicas

Veja a seguir as instruções básicas da linguagem procedural PL/pgSQL.

Atribuição

A atribuição de um valor a uma variável, ou a um campo de linha ou de registro, é escrita da seguinte maneira:

LINGUAGEM SQL



Código SQL.

Veja alguns exemplos:

LINGUAGEM SQL



Código SQL.

SELECT INTO

O resultado de um comando SELECT que apenas uma linha pode ser atribuída a uma variável registro, a uma variável tipo-linha ou a uma lista de variáveis escalares.

LINGUAGEM SQL



Código SQL.

O destino pode ser uma variável registro, uma variável linha, ou uma lista separada por vírgulas de variáveis simples e campos de registro/linha.

Veja um exemplo:

LINGUAGEM SQL



Código SQL.

Estruturas de controle

IF-THEN-ELSIF-ELSE

Veja a estrutura básica:

LINGUAGEM SQL



Código SQL.

Veja um exemplo:

LINGUAGEM SQL



Código SQL.

WHILE

A instrução **WHILE** repete uma sequência de instruções enquanto a expressão de condição for avaliada como verdade. A condição é verificada logo antes de cada entrada no corpo do laço. A sintaxe dessa instrução é a seguinte:

LINGUAGEM SQL



Código SQL.

Veja um exemplo:

LINGUAGEM SQL



Código SQL.

FOR

Cria um laço que interage em um intervalo de valores inteiros. A variável nome é definida automaticamente como sendo do tipo integer, e somente existe dentro do laço. As duas expressões que fornecem o limite inferior e superior do intervalo são avaliadas somente uma vez, ao entrar no laço. Normalmente, o passo da interação é 1, mas quando REVERSE é especificado se torna -1. A sintaxe básica é a seguinte:

LINGUAGEM SQL



Código SQL.

Alguns exemplos de FOR:

LINGUAGEM SQL



Código SQL.

Atividade 1

Qual é o objetivo da estrutura de repetição FOR no PL/pgSQL?

- A Para declarar variáveis dentro de um bloco de código PL/pgSQL.
- B Para executar uma única vez um conjunto de instruções SQL.
- C Para iterar sobre um conjunto de valores numéricos em uma sequência definida.
- D Para definir condições lógicas em uma cláusula WHERE de uma instrução SQL.
- E Para realizar operações matemáticas complexas em valores armazenados em variáveis.

Parabéns! A alternativa C está correta.

A estrutura de repetição FOR no PL/pgSQL itera sobre um conjunto de valores numéricos em uma sequência definida. É útil para executar um bloco de código repetidamente, percorrendo valores de 1 a N, por exemplo. Isso facilita a automatização de tarefas repetitivas e o processamento de dados em lotes.

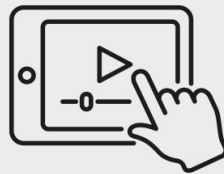
Procedimentos armazenados e funções

A Procedural Language/PostgreSQL (PL/pgSQL) serve para criar procedimentos armazenados que residem no servidor e funções que podem ser chamadas a partir de uma instrução SQL ou como a ação que um gatilho.

Você pode encontrar no arquivo [sqlmod4nc2](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos as estruturas dos procedimentos armazenados e das funções implementadas com a linguagem de programação procedural PL/pgSQL suportada pelo PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Procedimentos armazenados

O PostgreSQL 11 introduziu procedimentos armazenados que são subprogramas com suporte às transações. Para definir um novo procedimento armazenado, utiliza-se CREATE PROCEDURE, cuja sintaxe é a seguinte:

LINGUAGEM SQL



Código SQL.

Nessa sintaxe, respeite os passos a seguir:

- Especifique o nome do procedimento armazenado.
- Defina os parâmetros para o procedimento armazenado. Os parâmetros podem possuir os modos IN para entrada e INOUT para entrada e saída.
- Especifique PL/pgSQL como a linguagem de procedimento para o procedimento armazenado.
- Use \$\$ no início do código e após o END final.

Veja um exemplo:

LINGUAGEM SQL



Código SQL.

Consulte a tabela EMPREGADO a seguir:

Consulta à tabela EMPREGADO.

Para executar o procedimento aumentando os salários em 10%, utilize o seguinte comando:

Comando CALL.

Para ver o resultado, consulte novamente a tabela EMPREGADO.

Consulta à tabela EMPREGADO.

Funções definidas pelo usuário

Funções são um tipo de subprograma que retorna um valor em seu nome. A PL/pgSQL suporta dois tipos básicos de retorno: **escalar** ou **tabela**.

Funções com retorno escalar

A sintaxe para criar esse tipo de função é a seguinte:

LINGUAGEM SQL



Código SQL.

Veja um exemplo em que foi criada uma função que receba o ID de um empregado e retorne o salário anual correspondente, calculado a partir da multiplicação do salário do empregado por 13 e acréscimo 1/3 de férias.

Exemplo de função definida pelo usuário.

Para ver a execução, execute o seguinte comando:

Consulta à tabela EMPREGADO.

Funções com retorno tabela

A sintaxe para criar esse tipo de função é seguinte:

LINGUAGEM SQL



Código SQL.

Veja um exemplo em que a função recebe um cargo e retorna o ID e ULT_NOME do empregado.

Exemplo de função definida pelo usuário.

Para testar, execute o seguinte comando:

Exemplo de execução de função definida pelo usuário.

Eliminando funções

O comando que elimina funções apresenta a seguinte sintaxe:

LINGUAGEM SQL



Código SQL.

Onde CASCADE indica que devem ser eliminados os objetos dependentes da função, como gatilhos. O padrão é RESTRICT. Para eliminar a função F_EMP_RESUMO, o comando é o seguinte:

Exemplo de eliminação de função definida pelo usuário.

Atividade 2

Qual é o objetivo principal das funções no PL/pgSQL?

- A Definir regras de integridade referencial em um banco de dados PostgreSQL.
- B Criar índices que otimizam consultas em tabelas específicas.
- C Encapsular um conjunto de instruções SQL em uma unidade lógica e reutilizável de código.
- D Gerar automaticamente relatórios a partir de dados armazenados no banco de dados.
- E Fornecer interfaces gráficas de usuário para interação com o banco de dados PostgreSQL.

Parabéns! A alternativa C está correta.

As funções no PL/pgSQL têm como objetivo principal encapsular um conjunto de instruções SQL em uma unidade lógica e reutilizável de código. Isso permite que as operações complexas sejam realizadas de forma eficiente, simplificando o código, melhorando a manutenibilidade e promovendo a reutilização de lógica de programação. As funções podem aceitar parâmetros de entrada, processar dados e retornar resultados, tornando-as extremamente versáteis para uma variedade

de tarefas, desde cálculos simples até operações mais complexas de processamento de dados.

Função de gatilho

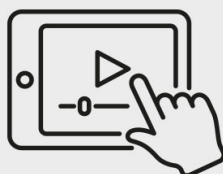
Funções são tipicamente subprogramas que podem ser chamados em um comando de SQL e que retornam um valor em seu nome.

Gatilhos são executados automaticamente em resposta a um comando DML em uma tabela. No PostgreSQL, a lógica do gatilho é criada em um tipo especial de função chamada **função de gatilho**.

Você pode encontrar no arquivo [sqlmod4nc3](#) os comandos utilizados nos exemplos.

Neste vídeo, abordaremos as estruturas dos gatilhos implementados com a linguagem de programação procedural PL/pgSQL suportada pelo PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Gatilhos

Um gatilho do PostgreSQL é uma função invocada automaticamente sempre que ocorre um evento associado a uma tabela. Um evento pode ser qualquer um dos seguintes: INSERT, UPDATE ou DELETE.

Para criar um gatilho, é preciso definir a função de gatilho e, em seguida, vincular essa função a uma tabela.

Tipos de gatilho do PostgreSQL

O PostgreSQL fornece dois tipos principais de gatilhos:

- Gatilhos em nível de linha.
- Gatilhos em nível de instrução.

As diferenças entre os dois tipos são quantas vezes o gatilho é invocado e a que horas. Por exemplo, ao emitir um UPDATE que modifica 20 linhas, o gatilho no nível da linha será invocado 20 vezes, enquanto o gatilho no nível da instrução será invocado 1 vez.

Invocação dos gatilhos

É possível especificar se o gatilho é acionado antes (BEFORE) ou depois (AFTER) de um evento.



Se o gatilho for acionado antes do evento, a operação na linha atual é cancelada ou a linha que está sendo atualizada ou inserida é modificada.



Se o gatilho for acionado após o evento, todas as alterações estarão disponíveis para o gatilho.

Criando a função do gatilho

Uma função de gatilho é semelhante a uma função normal definida pelo usuário, mas não possui parâmetros de entrada e tem um valor de retorno do tipo trigger.

A sintaxe é a seguinte:

LINGUAGEM SQL



Código SQL.

Uma função de gatilho recebe dados sobre seu ambiente de chamada por meio de uma estrutura especial, em que OLD.coluna e NEW.coluna representam os estados das colunas da linha na tabela antes ou depois do evento de disparo.

Depois de definir uma função de gatilho, é possível vinculá-la a um ou mais eventos de gatilho, como INSERT, UPDATE e DELETE.

Criando o gatilho

O comando que cria um gatilho possui a seguinte sintaxe:

LINGUAGEM SQL



Código SQL.

Onde você deve:

- Especificar o momento de disparo do gatilho, antes (BEFORE) ou depois (AFTER) da execução do comando.
- Especificar o evento que chama o gatilho. O evento pode ser INSERT, DELETE ou UPDATE.
- Definir a tabela à qual o gatilho deve ser associado.
- Determinar o tipo gatilho linha (ROW) ou instrução (STATEMENT).
- Especificar o nome da função de gatilho a ser executada quando do disparo do gatilho.

Exemplo de gatilho

Vamos criar um gatilho na tabela EMPREGADO que faça auditoria na alteração do salário do segundo a seguinte lógica: se o salário for alterado, devemos inserir na tabela: AUDIT_SALARIO ID do empregado, salário antigo, novo salário e data hora da alteração.

Vamos começar criando a tabela AUDIT_SALARIO:

Criação da tabela AUDIT_SALARIO.

Agora, vamos criar a função do gatilho:

Criação da função gatilho.

Em seguida, criamos o gatilho:

Criação do gatilho.

Para testar, damos um update de 1000 em todos os salários:

Teste da função gatilho.

Por fim, consultamos a tabela para ver as linhas inseridas:

Consulta do resultado da função gatilho.

Eliminando gatilho

O comando para eliminar um gatilho é o DROP TRIGGER:

Eliminando o gatilho.

Atividade 3

Qual é o propósito principal dos gatilhos (triggers) em um banco de dados?

A

Criar consultas complexas que envolvem múltiplas tabelas.

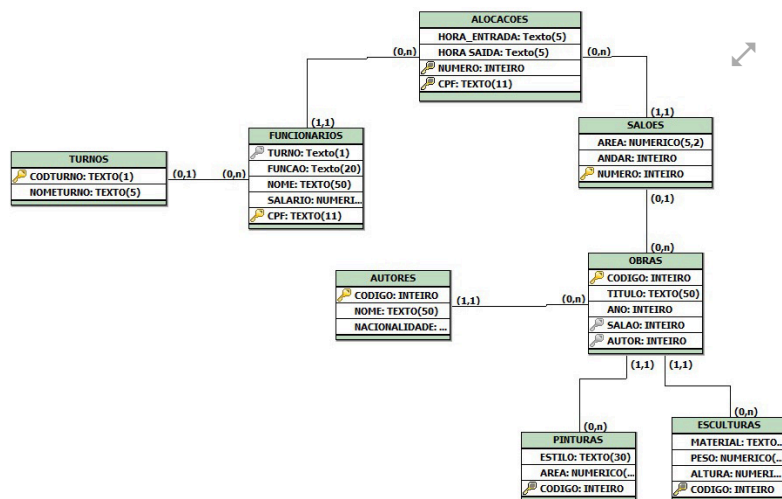
- B Definir restrições de integridade referencial entre tabelas.
- C Automatizar a execução de determinadas ações em resposta a eventos específicos no banco de dados.
- D Otimizar o desempenho das consultas utilizando índices.
- E Criar visualizações temporárias de dados para usuários específicos.

Parabéns! A alternativa C está correta.

Os gatilhos (triggers) em um banco de dados automatizam a execução de ações em resposta a eventos específicos, como inserção, atualização ou exclusão de registros em uma tabela. Eles permitem que os desenvolvedores definam instruções SQL que serão executadas automaticamente quando o evento ocorrer, sendo úteis para implementar lógica de negócios complexa, garantir a consistência dos dados e realizar auditorias, entre outros usos.

Procedimentos armazenados e PL/pgSQL

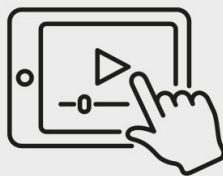
Veja alguns exemplos de uso dos comandos já aprendidos. Para isso, utilizaremos o banco de dados do museu, cujo modelo lógico é o seguinte:



Modelo lógico do banco de dados MUSEU.

Neste vídeo, abordaremos procedimentos armazenados implementados com a linguagem de programação procedural PL/pgSQL suportada pelo PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade execute os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta ao banco de dados do museu.

Após esses dois passos, realize as atividades descritas a seguir.

Criar a tabela MENSAGEM:

LINGUAGEM SQL



Código SQL.

Criar e executar um bloco PL/pgSQL que realize um LOOP FOR de 1 a 5 e inserir na tabela MENSAGEM o texto VOLTA X , onde X é o número da volta:

Consulta à tabela MENSAGEM.

Observe o código a seguir.

LINGUAGEM SQL



Código SQL.

A imagem a seguir vai exibir a consulta à tabela MENSAGEM.

Consulta à tabela MENSAGEM.

Repetir o exercício anterior contando de 5 A 1.

LINGUAGEM SQL



Código SQL.

Observe a imagem de consulta à tabela.

Consulta à tabela MENSAGEM.

Escrever um bloco que leia a quantidade de obras para uma variável e realizar um teste para determinar se existem muitas, regulares ou poucas obras:

LINGUAGEM SQL



Código SQL.

Veja também a imagem da consulta à MENSAGEM.

Consulta à tabela MENSAGEM.

As tabelas PINTURAS e ESCULTURAS possuem uma chave estrangeira para a tabela OBRAS, ao tentar excluir uma obra que seja referenciada por PINTURAS ou ESCULTURAS, ocorrerá um erro. Vamos escrever um procedimento armazenado que inicialmente irá eliminar a ESCULTURA ou PINTURA que faz referência à obra e depois fará a exclusão da obra propriamente dita.

Veja uma consulta à tabela obras antes da exclusão.

Consulta à tabela OBRAS.

Veja ainda a exclusão na tabela OBRAS.

Exclusão na tabela OBRAS.

Veja o código a seguir.

LINGUAGEM SQL



Código SQL.

Observe também a chamada a procedimento armazenado.

Chamada a procedimento armazenado.

Observe também a consulta à tabela OBRAS.

Consulta à tabela OBRAS.

Você pode baixar os códigos que resolvem os pedidos no arquivo [sqlmod4np4](#).

Atividade 4

Considere uma empresa de comércio eletrônico que lida diariamente com milhares de transações de clientes, armazenando informações sobre pedidos, inventário de produtos, clientes e outros dados cruciais.

Explique a relevância da inclusão de linguagens procedurais nos sistemas de gerenciamento de banco de dados (SGBD).

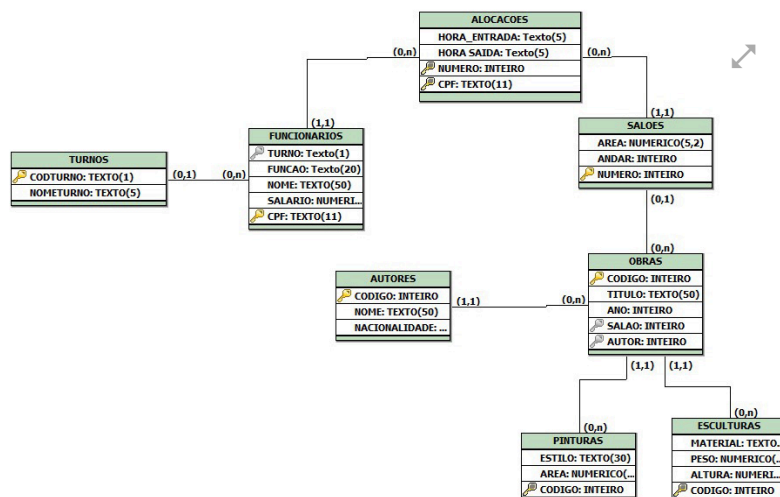
Digite sua resposta aqui

Chave de resposta ▾

As linguagens procedurais nos sistemas de gerenciamento de banco de dados (SGBD) são essenciais para a flexibilidade, eficiência e segurança das operações em um ambiente empresarial dinâmico e orientado por dados. Elas permitem o desenvolvimento de procedimentos armazenados e funções personalizadas diretamente no banco de dados para atender às necessidades específicas do negócio, além de fornecer recursos avançados de segurança com permissões granulares em procedimentos armazenados e funções.

Funções e gatilhos

Veja alguns exemplos de uso dos comandos já aprendidos sobre funções e gatilhos. Para isso, utilizaremos o banco de dados do museu, cujo modelo lógico é o seguinte:



Neste vídeo, abordaremos funções e gatilhos implementados com a linguagem de programação procedural PL/pgSQL suportada pelo PostgreSQL.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade execute os seguintes passos:

- Abra o PGAdmin4 e faça conexão no servidor.
- Abra uma janela de consulta ao banco de dados do museu.

Em seguida, realize as atividades descritas a seguir:

Escrever os códigos solicitados:

- Escrever um gatilho para criar uma trilha de auditoria.
- Considerar que o gatilho deve disparar quando for alterado o autor de uma obra. Ele deverá inserir um registro na tabela `AUDIT_AUTOR` com o código da obra, o antigo autor e o novo autor.
- Criar primeiro a tabela de auditoria.

Veja como fica o código.

LINGUAGEM SQL



Código SQL.

Criar a função de gatilho:

LINGUAGEM SQL



Código SQL.

Criar o gatilho:

LINGUAGEM SQL



Código SQL.

Alternar o autor da obra de código 102 do AUTOR 102 para o AUTOR 101:

LINGUAGEM SQL



Código SQL.

Verificar o efeito do gatilho:

Consulta à tabela AUDIT_AUTOR.

Escrever um gatilho para melhorar a mensagem de erro retornada para o usuário. Perceba que a tabela OBRAS possui a chave estrangeira AUTOR para AUTORES. Ao tentar excluir um autor que possui obras, receberá uma mensagem de erro.

Exclusão na tabela OBRAS.

A mensagem de erro não é muito clara para um usuário comum, então inicialmente escreva uma UDF que retorne um valor escalar com a quantidade de obras que o autor possui.

LINGUAGEM SQL



Código SQL.

Testar a função:

Teste da função definida pelo usuário.

Escrever a função de gatilho:

LINGUAGEM SQL



Código SQL.

Criar o gatilho:

LINGUAGEM SQL



Código SQL.

Testar o gatilho:

Teste do gatilho.

As tabelas PINTURAS e ESCULTURAS possuem uma chave estrangeira para a tabela OBRAS. Se você tentar excluir uma obra que seja referenciada por PINTURAS ou ESCULTURAS, ocorrerá um erro. É preciso escrever um gatilho que, inicialmente, irá eliminar a ESCULTURA ou PINTURA que faz referência à obra e, em seguida, fará a exclusão da obra propriamente dita.

Tentar eliminar:

Tentativa de eliminação de registro na tabela OBRAS.

Criar a função de gatilho.

LINGUAGEM SQL



Código SQL.

Criar o gatilho:

LINGUAGEM SQL



Código SQL.

Testar o gatilho:

Teste do gatilho.

Teste do gatilho.

Você pode baixar os códigos que resolvem os pedidos no arquivo [sqlmod4np5](#).

Atividade 5

Considere um cenário em que a empresa possui um sistema de gerenciamento de pedidos on-line, no qual os clientes podem fazer pedidos de produtos e os funcionários podem processar esses pedidos.

Nesse contexto, explique como os gatilhos desempenham um papel crucial na garantia da integridade dos dados e na aplicação das regras de negócio.

Digite sua resposta aqui

Chave de resposta ▾

Os gatilhos em um sistema de gerenciamento de banco de dados desempenham um papel crucial na implementação de regras de negócio específicas. Um gatilho pode garantir que um pedido só possa ser marcado como "enviado" se todos os itens estiverem disponíveis em estoque. Isso é alcançado verificando a disponibilidade de estoque sempre que uma atualização é feita na tabela de pedidos. Se algum item estiver em falta, o gatilho pode impedir a atualização do status do pedido, garantindo que os pedidos só sejam marcados como "enviados" quando todos os itens estiverem prontos para envio.

O que você aprendeu neste conteúdo?

- Como modelar um banco de dados.
- Como criar as tabelas e manipular os dados de um banco em um SGBD.
- Como consultar dados utilizando a linguagem SQL.
- Como criar procedimentos armazenados, funções e gatilhos utilizando uma linguagem procedural de banco de dados.

Explore +

No site oficial do PostgreSQL, você encontra a documentação completa do sistema. Consulte!

Referências

DOCUMENTAÇÃO Oficial do PostgreSQL. Consultado na internet em: 27 mar. 2024.

BALIEIRO, R. **Banco de dados**. Rio de Janeiro: SESES, 2015.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de banco de dados**. 5. ed. Rio de Janeiro: Campus, 2006.

Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material

O que você achou do conteúdo?



 Relatar problema